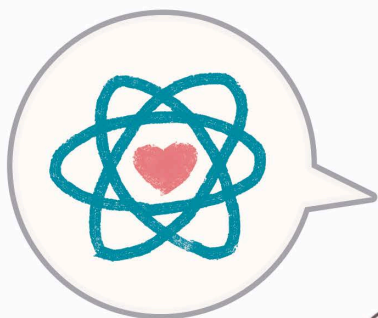


いあウト!



第5版
React 19

TypeScript
で始める
つらくない
React 開発

Sample

大岡由佳

② React 基礎編

最新の React 19 に完全対応

思想・歴史から React を読み解く
初学者の「Why?」に答えてくれる本

シリーズ累計

3.5万部
突破!

読者
からの声

「説明がめちゃくちゃ丁寧!」
「この本を最初に読みたかった……」

BOOTH
技術書部門

1位

**りあクト！ TypeScript で始めるつらくない
React 開発 第5版 【② React 基礎編】**

本書について

登場人物の紹介

柴崎雪菜（しばさき ゆきな）

都内のとあるインターネットサービスを運営する会社のテックリード・エンジニア。2016 年ごろから React を使っている。本格的なフロントエンド開発チームを作るための中核的人材として、今の会社に転職してきた。チームメンバーを集めるため人材採用にも関わり自ら面接も行っていたが、彼女の要求水準の高さもあってなかなか採用に至らない。そこで「自分が React を教えるから他チームのエンジニアを回してほしい」と上層部に要望を出し、社内公募が行われた。

秋谷香苗（あきや かなえ）

柴崎と同じ会社に勤務する、新卒 2 年目のやる気あふれるエンジニア。入社以来もっぱら Ruby on Rails によるサーバサイドの開発に携わっていたが、柴崎のメンバー募集に志願してフロントエンド開発チームに参加した。そこで柴崎から「1 週間で戦力になって」といわれ、彼女にマンツーマンで教えるを受けることになる。

前版との差分および正誤表

過去の版からの変更点と、本文内の記述内容の誤りや誤植についての正誤表は以下のページに掲載しています。なお、電子書籍版では訂正したものを新バージョンとして随時配信していきます。

- 各版における内容の変更
<https://github.com/klemiary/Riakuto-StartingReact-ja5.0/tree/main/CHANGELOG.md>
- 『りあクト！TypeScript で始めるつらくない React 開発第 5 版』正誤表
<https://github.com/klemiary/Riakuto-StartingReact-ja5.0/tree/main/errata.md>

サンプルコードについて

本文で紹介しているサンプルコードは、GitHub に用意した下記の専用リポジトリにて、章・節ごとに分けてすべて公開してあります。

<https://github.com/klemiuary/Riakuto-StartingReact-ja5.0>

挙動をその場で確認していただくため StackBlitz に置いているサンプルもありますが、本文中に随時 URL を掲載していますのでそちらをブラウザでご覧ください。学習を効率的に行うためにも、本文を読み進めながらこれらのコードを実際に実行することを強くおすすめします。またコードをご自身で変更して挙動のちがいを確かめてみるのも理解の助けになるはずです。

なおリポジトリのコードと本文に記載されているコードには、ときに差分が存在します。紙面に適切になるよう改行位置を調整したり、各種コメントアウト文を省略するなどによるものですが、そのため行番号が一致しないことがありますので、ご注意ください。

本文および上記リポジトリに掲載しているコードは、読者自身のプログラムやドキュメントに流用してかまいません。それにあたりコードの大部分を転載する場合を除き、筆者に許可を求める必要はありません。出典を明記する必要はありませんが、そうしていただければありがたいです。

本書内で使用している主なソフトウェアのバージョン

• React (react)	19.1.0
• React DOM (react-dom)	19.1.0
• Vite (vite)	6.3.5
• TypeScript (typescript)	5.8.3
• Tailwind CSS (tailwindcss)	4.1.6
• shadcn/ui (shadcn)	2.5.0
• ESLint (eslint)	9.26.0
• typescript-eslint (typescript-eslint)	8.30.1
• Prettier (prettier)	3.5.3
• Lefthook (lefthook)	1.11.12

目次

本書について	2
登場人物の紹介	2
前版との差分および正誤表	2
サンプルコードについて	3
本書内で使用している主なソフトウェアのバージョン	3
第5章 JSX で UI を表現する	8
5-1. なぜ React は JSX を使うのか	8
5-1-1. JSX はどのように変換されるか	8
5-1-2. なぜ React ではデザインとロジックを混在させるのか	12
5-1-3. なぜ React は HTML テンプレートを使わないのか	15
5-1-4. JSX は汎用的に UI を表現する	20
5-2. JSX 構文の書き方	23
5-2-1. JSX の基本的な文法	23
5-2-2. JSX とコンポーネントの関係	29
5-2-3. React の組み込みコンポーネント	34
第6章 React をめぐるモダン Web 開発の歴史（前編）	42
6-1. 第 1 世代：Ajax と DOM 操作のためのツールキット	42
6-2. 第 2 世代：SPA フレームワークの登場	44
6-3. パラダイムシフトを起こした React	49
6-3-1. React 誕生秘話	49
6-3-2. React は何を革新したのか	51
第7章 ビルドツールでプロジェクトを運用する	58
7-1. React 開発におけるビルドツールの変遷	58
7-1-1. React の開発環境を作るのに必要なツール	58
7-1-2. webpack 帝国の完成	60
7-1-3. 新世代ビルドツールの台頭	64
7-1-4. Vite の時代	67
7-1-5. Vite のライバルとビルドツールの将来	71
7-2. Vite で開発環境を構築する	76
7-2-1. create-vite によるセットアップ	76
7-2-2. Vite 環境のカスタマイズ	80
7-2-3. 拡張子、書くべきか書かざるべきか	85
第8章 コンポーネントの基本を学ぶ	87

8-1. UI コンポーネントライブラリの導入.....	87
8-1-1. UI コンポーネントライブラリとは.....	87
8-1-2. コンポーネントのスタイリング戦略.....	89
8-1-3. shadcn/ui の環境を構築する.....	94
8-2. コンポーネントに Props を受け渡す.....	97
8-3. コンポーネントに状態を持たせる.....	106
8-4. コンポーネントで副作用を扱う.....	112
8-5. React におけるコンポーネント構文の変遷.....	117
8-5-1. 最初はクラスだったコンポーネント.....	117
8-5-2. 関数コンポーネントと Hooks.....	122
8-6. コンポーネントからロジックを分離し、再利用できるようにする.....	126
第9章 リンターとフォーマッターでコード美人に.....	131
9-1. リンターでコードの書き方を矯正する.....	131
9-1-1. リンターの選定.....	131
9-1-2. ESLint の設定記法.....	134
9-1-3. Flat Config で ESLint の設定を書く.....	138
9-1-4. ESLint の環境をさらにカスタマイズ.....	144
9-1-5. ESLint の設定内容を確認する.....	155
9-2. フォーマッターでコードを整形する.....	159
9-2-1. フォーマッターの地位を確立した Prettier.....	159
9-2-2. Prettier の環境を作る.....	162
9-3. Git Hooks と連携して実行させる.....	166
第10章 React をめぐるモダン Web 開発の歴史（後編）.....	172
10-1. 第3世代：SPA モデル黄金時代.....	172
10-1-1. React 登場以後の世界.....	172
10-1-2. Angular.....	173
10-1-3. Vue.js.....	176
10-1-4. Svelte.....	179
10-2. 第4世代：メタフレームワークの時代.....	182
10-2-1. バック・トゥ・ザ・サーバ.....	182
10-2-2. React の上に構築されたフレームワーク.....	186
10-2-3. Next.js.....	190
10-2-4. Remix（React Router v7 以降）.....	197
10-2-5. どのメタフレームワークを採用するべきか.....	206

他巻目次

「① 言語・環境編」目次

第1章 こんにちは React

- 1-1. 基本環境の構築
- 1-2. React のプロジェクトを作成する
- 1-3. プロジェクトを管理するためのコマンドやスクリプト

第2章 ライトでディープな JavaScript の世界

- 2-1. あらためて JavaScript ってどんな言語？
- 2-2. 変数の宣言
- 2-3. JavaScript のデータ型
- 2-4. 関数の定義
- 2-5. クラスを表現する
- 2-6. 配列やオブジェクトの便利な構文
- 2-7. 式と演算子で短く書く
- 2-8. モジュールを読み込む
- 2-9. それでも知っておくべき JavaScript の this

第3章 関数型プログラミングでいこう

- 3-1. 関数型プログラミングは何がうれしい？
- 3-2. コレクションの反復処理
- 3-3. JavaScript で本格関数型プログラミング
- 3-4. 非同期処理と例外処理

第4章 TypeScript で型をご安全に

- 4-1. 時代が望んだ言語 TypeScript
- 4-2. TypeScript の基本的な型
- 4-3. 関数とクラスの型
- 4-4. 型の名前と型合成
- 4-5. さらに高度な型表現
- 4-6. 型アサーションと型ガード

4-7. モジュールのインポート／エクスポート

4-8. TypeScript の環境設定

「③ React 実践編」 目次

第 11 章 React のページをルーティングする

- 11-1. React におけるルーティングとは
- 11-2. React 用のルーティングライブラリ
- 11-3. React Router を使う
- 11-4. Next.js で App Router を使う

第 12 章 外部リソースのデータを取得する

- 12-1. 各種の外部データ取得手法
- 12-2. Fetch-on-Render アプローチ
- 12-3. Fetch-then-Render アプローチ
- 12-4. Suspense for Data Fetching

第 13 章 エラーをハンドリングする

- 13-1. Error Boundary とは
- 13-2. react-error-boundary を使う
- 13-3. Next.js の App Router でエラーハンドリング
- 13-4. React Router（フレームワーク）でエラーハンドリング

第 14 章 メモ化でレンダリングを効率化する

- 14-1. 計算結果をメモ化する
- 14-2. 関数定義をメモ化する
- 14-3. React Compiler を導入する

第 15 章 フォームでデータを更新する

- 15-1. React でフォームの UI を実装するには
- 15-2. フォームを Action でハンドリングする
- 15-3. フォームライブラリを利用する

第5章 JSX で UI を表現する

5-1. なぜ React は JSX を使うのか

5-1-1. JSX はどのように変換されるか

「さて次は JSX だね。秋谷さんは JSX についてどれくらいのことを知ってる？」

「JSX って React に標準搭載されてるテンプレート言語ですよ。Rails に対する ERB¹ のような」

「.....うーん、ちょっと誤解があるようだね。じゃあまずその誤解を解消するためにも、JSX とは実際のところ何なのかを説明しておこう。JSX という名前の由来は『JavaScript Syntax Extension』とも『JavaScript XML』ともいわれる。つまり JSX は JavaScript の中に XML ライクな記述ができるようにした**構文拡張**なの」

「その『構文拡張』というのは？」

「言語標準には含まれない特殊な用途のための便利な構文を、後づけで使えるようにしたもののこと。実際には JSX は標準の JavaScript にコンパイルされることを前提としてる。たとえばこの JSX コードは、現在の React のデフォルト設定では次のようにコンパイルされる」

リスト1: コンパイル前の JSX コード

```
<button type="submit" autoFocus>
  Click Here!
</button>
```

リスト2: コンパイル後の JavaScript コード

```
import { jsx as _jsx } from 'react/jsx-runtime';

_jsx('button', {
  type: 'submit',
  autoFocus: true,
  children: 'Click Here',
});
```

¹ 「Embedded Ruby」の略で、「eRuby」とも。HTML へ Ruby スクリプトを埋め込むためのテンプレートシステム。Ruby 1.8 以降に標準ライブラリとして組み込まれており、Ruby on Rails の view を記述するための開発言語として標準採用されている。

「タグが展開されて `_jsx()` という関数に要素名や各種属性を渡す形になってますね」

「この変換を実際に行ってるのは後で説明する Babel や tsc などのコンパイラなんだけど、実はそのコンパイル結果はオプション設定によって複数の様式に分かれる。TypeScript の場合、コンパイラオプション `jsx` がその設定ね。設定値によって出力は次のようになる²⁾」

設定値	入力	出力	出力ファイルの拡張子
<code>preserve</code>	<code><div /></code>	<code><div /></code>	<code>.jsx</code>
<code>react</code>	<code><div /></code>	<code>React.createElement("div");</code>	<code>.js</code>
<code>react-native</code>	<code><div /></code>	<code><div /></code>	<code>.js</code>
<code>react-jsx</code>	<code><div /></code>	<code>_jsx("div", {}, void 0);</code>	<code>.js</code>
<code>react-jsxdev</code>	<code><div /></code>	<code>_jsxDEV("div", {}, void 0, false, {...}, this);</code>	<code>.js</code>

「こんなに種類があるんですか。さっきのコンパイル結果はこのオプション値が `react-jsx` と設定されていた場合のものなんですね」

「そのとおり。なおこの設定は 2020 年 10 月にリリースされた React 17.0 で導入された変換形式³⁾、TypeScript もバージョン 4.1 でそれに対応したという経緯がある。それ以前は長らくこの設定値のデフォルトは `react` で、コンパイル結果はこうなっていた」

リスト3: 旧来の JSX トランスフォーム

```
React.createElement(
  'button',
  { type: 'submit', autoFocus: true },
  'Click Here!'
);
```

「さっきの結果と比べてほしいんだけど、こちらの結果ではインポートしてないはずの `React` オブジェクトがいきなり使われてる。だからこの設定下では、JSX 構文を使うときはファイルの先頭で明示

²⁾ 「TypeScript: Documentation - JSX」
<https://www.typescriptlang.org/docs/handbook/jsx.html>

³⁾ 「新しい JSX トランスフォーム - React Blog」
<https://ja.legacy.reactjs.org/blog/2020/09/22/introducing-the-new-jsx-transform.html>

的に `import React from 'react'` を書いてあげないとコンパイルできない。だから昔の React のコードでは、JSX ファイルの 1 行めにはこれが必ず書かれてた」

「面倒だったんですねえ。ちなみに React が JSX の変換形式を刷新した理由って何だったんですか？」

「React 開発チームの動機としては、JSX の変換をコンパイラの中で完結させて React に依存しないようにしたかったみたい⁴。またパッケージ構造も JSX 部分の実装を `react/jsx-runtime` に切り分けられるようになった。さらに新形式のほうが処理を最適化しやすくパフォーマンスも多少は向上するらしい」

「なるほど」

「ただ JSX の本質を理解するには従来の変換形式のほうが都合がいいので、これ以降の説明は従来形式を使うことにするね。その場合 JSX 構文は基本的に、XML のノードツリーを `React.createElement`⁵ というメソッドのコールに変換してる。そしてその結果、次のようなオブジェクトが生成されるの」

```
{
  type: 'button',
  props: {
    type: 'submit',
    autoFocus: true,
    children: ['Click Here'],
  },
  key: null,
};
```

「これは **React Elements** といって React アプリケーションを構成する最小の単位であり、TypeScript の環境では `ReactElement` というインターフェースが適用されたオブジェクトになる。つまり React において JSX の構文は、本質的には `React.createElement(...)` のシンタックスシュガーであり、`ReactElement` オブジェクトを生成する特殊な式だと言えるの」

「シンタックスシュガー……、式……」

「秋谷さんが最初に口にした『JSX はテンプレート言語』という表現がまちがってるのがそこだね。JSX は最終的に JavaScript でオブジェクトを生成する式に還元されるものであって、JavaScript の枠組みを超えた別体系のものじゃない。だからこそ変数に代入したり、狭義のオブジェクトのプロパティ

⁴ 「rfcs/0000-create-element-changes.md at createelement-rfc • reactjs/rfcs」
<https://github.com/reactjs/rfcs/blob/createelement-rfc/text/0000-create-element-changes.md>

⁵ 「createElement – React」 <https://ja.react.dev/reference/react/createElement>

値にしたり、関数の引数や戻り値にしたりもできるわけ。こんなふうだね」

```
const foo = <span>Hello!</span>;
const obj = { bar: foo };
const fn = (elem: React.ReactElement) => <div>{elem}</div>;
```

「.....あー、なんかわかってきたかもしれない。ただこの書き方、ちょっと気持ち悪いですね。どうしても JSX のところをクォーテーションで囲いたくなっちゃう」

「その『気持ち悪い』というのは最初に JSX を見た人がよく口にする感想なんだけど、本質がわかっていけば後は単なる慣れの問題だよ。JSX と同じく DOM 要素を生成する拡張構文を提供するライブラリに Hyperscript⁶ というのがあるんだけど、こっちは JavaScript 関数を直接使う構文になってる」

```
const element = h('div#main', [
  h('h1', 'The Title'),
  h('p', [
    'Hello, I am writing this text with ',
    h('strong', 'Hyperscript'),
  ]),
]);
```

「Hyperscript は JSX のように『気持ち悪い』とデイスられることはなく、またコンパイルを必要としない軽いライブラリだったのもあって一時はいくつものフレームワークに採用されたんだけど、今ではほとんど使われなくなってしまった。けっきょく論理的な整合性よりも、みんな直感的で読み書きしやすいほうに落ち着くんだよ」

「うーむ、たしかにそうかもしれませんね」

「それに JSX は React 固有のものだと思われがちだけど、React から独立した仕様として公開されていて⁷、他のフレームワークでも使えるようになってる。Vue.js なんか公式ドキュメントに JSX を使うための説明をわざわざ入れてるしね⁸」

「へー、Vue でも JSX が使えるんですね。知らなかった」

「React では JSX はコンポーネントの View を記述するために使われる。コンポーネントは関数で表現

⁶ <https://hyperscript.org/>

⁷ <https://facebook.github.io/jsx/>

⁸ 「レンダー関数と JSX | Vue.js」 <https://ja.vuejs.org/guide/extras/render-function>

され、その戻り値として JSX が返されるようにする。雰囲気を知ってもらうために、JSX を使ったコンポーネントの記述をチラ見せするとこんな感じのコードになるかな」

```
import useSWR from 'swr';

export default function UserList() {
  const url = 'https://jsonplaceholder.typicode.com/users';
  const fetcher = () => fetch(url).then((res) => res.json());
  const { data: users, error, isLoading } = useSWR('users', fetcher);

  if (isLoading) {
    return <div>Now loading...</div>;
  } else if (error) {
    return <div className="error">Sorry! Failed to load</div>;
  }

  return (
    <ul>
      {users.map((user) => (
        <li key={user.id}>
          {user.id}: {user.name} ({user.email})
        </li>
      ))}
    </ul>
  );
}
```

「むむむむ……。最初に『Hello, World!』をやったときは単純な処理だったから気にならなかったんですが、React ってデザインとロジックのコードがごちゃまぜになってますよね？ Rails では Model と Controller にロジックを書き、View のテンプレートで見た目を書くというふうにきっちり分かれてたので、そこにすごく違和感があります」

「サーバサイドアプリケーションの開発から入ってきた人は、たいていそんなふうに言うね。じゃあなぜ React ではデザインとロジックの記述が混在する形になってるのか、次はそれを説明しようか」

5-1-2. なぜ React ではデザインとロジックを混在させるのか

「Ruby on Rails に代表されるサーバサイド Web アプリケーションフレームワークは、そのほとんどがアーキテクチャに MVC のパターンを採用してる。MVC とはアプリケーションを Model、View、Controller の 3 つの要素に分割して構築する手法。Web アプリケーションにおいては、ひとつの URL

リクエストに対してそれを受けた Controller が起点となり、Model を操作して必要なデータの取得・加工を行って、最後に View でそのデータをテンプレートへ埋め込んでページを出力するというフローになる。図で示すと次のようになるかな」

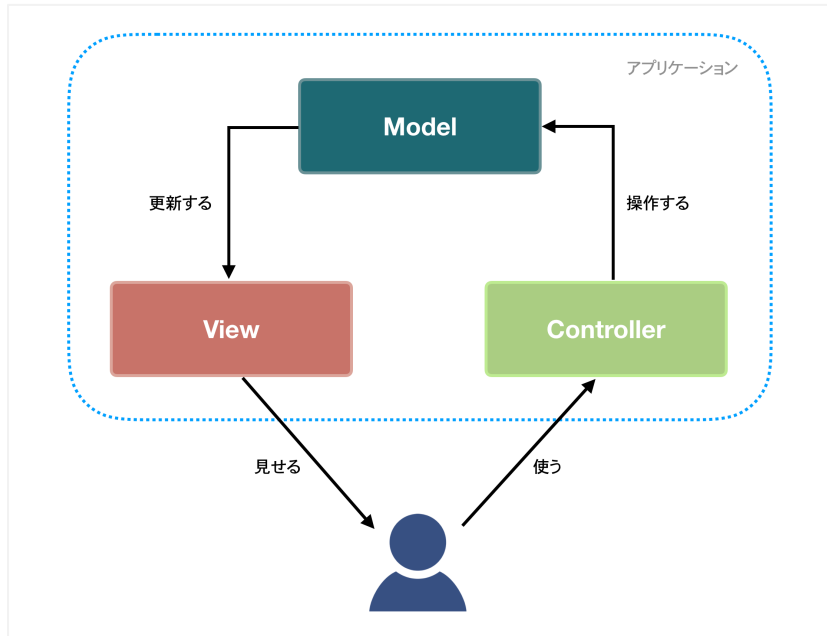


図 1: MVC アーキテクチャの概念図

「はい、Rails エンジニアだった私にはなじみのある図ですね」

「ところでソフトウェア工学において、アーキテクチャとかデザインパターンというのは**関心の分離**を行うためにあるものなのね。MVC ではどんな関心によって要素を区分しているかという、アプリケーション全体で**技術的な役割**によってその3つに区分してる」

「ふーむ、まあそのとおりですね」

「この技術的な役割によって関心を分離するというのは一時代を築いた考え方だけど、だからといってそれが普遍的な真理というわけじゃない。特にフロントエンド開発ではその歴史の中で、MVC パターンはうまく機能しないという見解が開発者の間で主流になってるの」

「ええっ？ そうなんですか？」

「React においては、関心の分離の単位が MVC のように技術的な役割じゃない。アプリケーションにおけるひとつひとつの**機能**こそがその単位になってる。それぞれの機能単位で分割された独立性の高いパーツをレゴブロックのように組み合わせることでアプリケーションを構築しようというのが React の開発思想であり、そのパーツに相当する概念こそが**コンポーネント**というわけ。独立した機能単位のパーツとして分割するためには、そのパーツの中に UI デザインとロジックを閉じ込める必

要がある。そうじゃないとパーツ同士が疎結合にならないでしょ」

「むむむむ……」

「たぶん Rails しか知らない秋谷さんには、React におけるコンポーネントの独立性というのがピンと来てないんだと思う。Rails だとそのページの表示に必要な処理を Controller が全部終わらせてから、View で常にページ全体をまとめてレンダリングする。でも React ではページを構成する部品であるコンポーネントが必要な処理を自律的に行い、レンダリングされるかどうか個別に判断される」

「パーツごとの個別のレンダリング……。そういえば X (Twitter) とかは、ページ全体のレイアウトはすでに表示されてるのに、部分的にデータの取得待ちでローダーがぐるぐる回って後から遅れて表示されますよね。あれもそうですか？」

「そう、Web 版の X はまさに React 製だから。各コンポーネントが自律的にサーバ API と通信してデータを取得し、個別に再レンダリングされるからああいう挙動になるの。概念図にするとこんな感じかな」

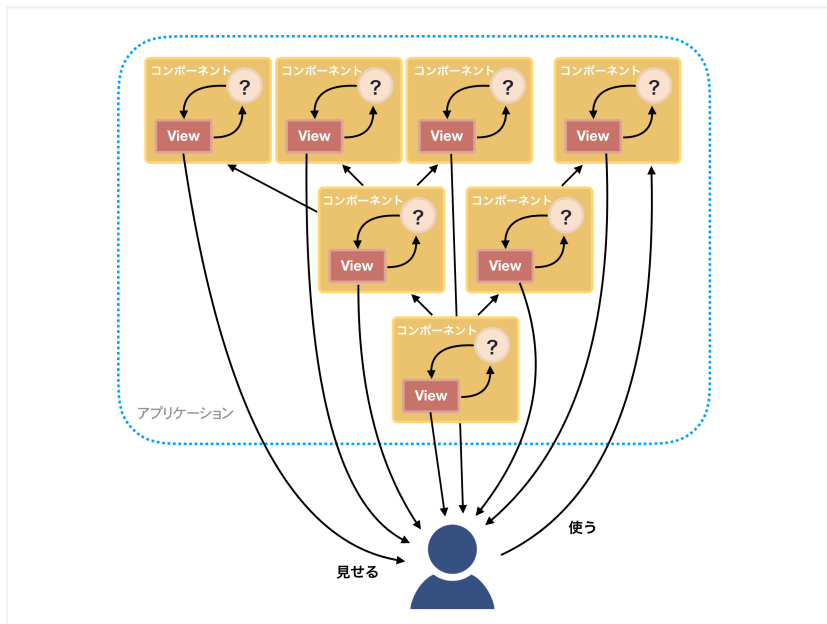


図 2: React のコンポーネントベースなアーキテクチャの概念図

「そしてコンポーネントのロジック、たとえば表示のためのデータをどうやって取得するかとか、起こったイベントをどう処理するかとか、状態の変化が自身の表示にどう影響するかとかなどは、そのコンポーネントのレンダリングと本質的に結合したものであって、それを技術の役割単位で分割してしまうことのほうが無理があると React では考えるのね。これはまあ、実際に開発してみないと実感

できないことかもしれないけど」

「うーん、でも開発者はそれでいいかもしれませんが、じゃあデザイナーとの分業はどうなるんですか？ Rails による開発ではデザイナーさんに ERB のテンプレートを書いてもらってたんですけど、このロジックが混在した JSX のコードをデザイナーさんが書くのは無理じゃないでしょうか？」

「それは問題の立て方がそもそもおかしいんだよ。モバイルアプリの開発で同じことをデザイナーにしてもらおう？ デザイナーが作ってくれるのはサイズやレイアウトが確認できる Figma や Sketch のデザインキャンパスまででしょ。アプリの開発者はそれを参考にしながら自分で UI を組み上げていく」

「ええっ、それはそうですけど。……うーん」

「実際、モダンなフロントエンド主体のアプリケーション開発の現場ではデザイナーが HTML や CSS のコーディングまで行うことは稀^{まれ}だね。純粋に UI や UX デザインを専門にしている HTML や CSS が書けない人もめずらしくない」

「じゃあ、HTML と CSS のコーディングも私たちがやるんですか？」

「そうだよ、それもエンジニアの仕事。今は Figma から HTML や CSS を書き出すプラグインもあるし、React 用の汎用的な UI ツールキットも充実してるので、エンジニアが UI の実装まで担当するのが一般的なの」

「ううっ、そうはいつでもこれまで自分でまともに CSS を書いたことなかったんで……。React エンジニアへの道は険しいなあ」

「フロントエンドフレームワークは、モバイルアプリのようなリッチでインタラクティブな UI を Web で提供することを目的に作られてるからね。同じ Web アプリケーションといっても Ruby on Rails のようなサーバサイドフレームワークとはパラダイムが異なる。だから必然的に、仕事の分担もそれに適合したものに变化していくの」

5-1-3. なぜ React は HTML テンプレートを使わないのか

「React のアーキテクチャがコンポーネントをベースにしているという話をしたけど、コンポーネント指向の React の登場がそれまでのフロントエンド開発の問題をあまりにも鮮やかに解決したために、現在のほとんどのフロントエンドフレームワークはその影響を受けてコンポーネントベースになってしまった。ただその中には、アプリケーションをコンポーネント単位に分割しながらもその View のレンダリングに HTML テンプレートを用いているフレームワークも少なくない」

「そういえば Vue はテンプレートを使いますもんね」

「そう。それらは Angular のようにテンプレートを専用ファイルとして分離するか、Vue.js の SFC (Single-File Components、単一ファイルコンポーネント)⁹のように特別なタグでくくって記述するか

⁹ 「単一ファイルコンポーネント | Vue.js」 <https://ja.vuejs.org/guide/scaling-up/sfc>

第6章 React をめぐるモダン Web 開発の歴史（前編）

6-1. 第1世代：Ajax と DOM 操作のためのツールキット

「さて、ここまででモダン JavaScript およびそれを用いた関数型プログラミング、TypeScript による型安全なプログラミング、そして JSX の文法までを学んだ。早く実際の React アプリケーションを作ってみたいところだろうけど、その前に知っておいてもらいたいことがある。

React 初心者が学習を進めていくと『React はなぜこんな難しくて回りくどい書き方をするのか』という不満を含んだ疑問をしばしば抱きがち。それらを決まりだからこうなんだと押し付けられても、納得できない身につきにくいでしょ。ただ初心者が思いつきそうな疑問には、歴史の中で一度は試されつつも淘汰されてしまったものが少なくない。そういった試行錯誤の末に今の React が形作られてきたことを知っていれば、学習の大きな助けになるはず。だから実際にアプリケーションのコードを書き始める前に、React を生んだ思想と歴史について説明しておきたいの」

「これまでもそうしてくれてましたよね。すごくわかりやすかったので、ぜひお願いしたいです！」

「じゃあ長くなるけどその創成期から話そうか。Web アプリケーションは 1993 年の CGI⁴¹ の登場以来ずっと、ブラウザからのリクエストを受けてサーバが必要な処理を行って結果を返すという、サーバサイド主体のものだった。処理系の主役が Perl (CGI) → Java (サーブレット⁴²) → PHP

(mod_php⁴³) のように変遷していったけれども、サーバサイドに処理の主体があってクライアント（ブラウザ）はそれを受け取るだけという形式は当たり前のものとして続いていたの」

「Rails もその形式のフレームワークですよ」

「そうだね。Ruby on Rails の最初のバージョン 0.5 が公開されたのは 2004 年のこと。『15 分でブログサイトが作れる』『Java の 10 倍の開発生産性』というのは多少大げさな触れ込みだったけど、DX（開発者体験）の改善を重視したフレームワークの元祖と言えるだろうね。

そんな Rails も開発者にとっては画期的だったけれども、それが実現する Web アプリケーションの UX（ユーザー体験）については従来と何ら変わらなかった。しかし翌 2005 年に、世界を革新する歴史的な事件が起きる。それが **Google マップ** のリリース。Google マップはマウスのドラッグによっ

⁴¹ Common Gateway Interface。クライアントのブラウザの要求に応じて、Web サーバが外部プログラムを別プロセスで実行し、その実行結果を送信するしくみのこと。

⁴² サーバ上で Web ページを動的に生成したりデータ処理を行うために、Java で作成されるプログラムのこと。その実行環境であるサーブレットコンテナとして Apache Tomcat などがある。

⁴³ Apache HTTP Server のモジュールとして動作する PHP の処理系。PHP 登場初期はこの形式で運用するケースが多かった。

て、見る位置を好きなように移動させたり、拡大・縮小させたりというのがリアルタイムでできた」
「今では当たり前に思えますけど、その何が革新的だったんでしょうか？」

「たとえば当時の Yahoo! 地図情報は地図の周囲に 8 方向の矢印が表示されていて、それをクリックするとページが遷移してそこで一コマ隣の領域が表示される。同様にズームにもページ遷移が必要で、そのたびにページ全体の読み込みを待たされる非常にストレスがたまるものだったの。そんな苦行のような地図の閲覧が、マウス操作でリアルタイムかつ直感的な操作で行えるようになったのは当時のインターネットユーザーには衝撃だった。だから特に用がなくても Google マップをぐりぐり動かして遊んだものよ」

「あ、柴崎さんもそうやって遊んだんですね……」

「Google マップは UX の点で革新的だったけど、開発者に与えた衝撃も大きかった。Flash などのプラグインを使うことなく、ブラウザに処理系が標準搭載されている JavaScript のみでそれが実現されていたから。それまで JavaScript はホームページを無意味にキラキラと飾り付けるための言語くらいにしか思われてなかったのが、こんなに本質的で高度なことができるのかと再評価のきっかけになった。その中核となったのが、ブラウザで非同期通信をしながら動的にページを書き換える Ajax (Asynchronous JavaScript + XML) と呼ばれる技術ね」

「エイジャックス……。かすかに聞いたおぼえがあります」

「当時はブラウザ間の競争が激しく、各社先駆けてさまざまな機能を実装してデファクトスタンダードになることを狙ってた。その中で Internet Explorer が 1999 年に最初に実装し、Netscape や Safari が後追ひした XMLHttpRequest (XHR) という機能があったの。これはサーバと XML フォーマットでデータを通信するための組み込みオブジェクトで、すでに読み込んだページから HTTP リクエストを行うことを可能にし、ページ遷移することなしにデータを送受信できるものだった。Outlook で使われていたというこの XHR は、その後ほぼ忘れられていたのを Google のチームが再発見してマップの開発に活用したわけね。JavaScript から XHR による非同期通信を利用し、動的にページの一部を書き換えるアプローチが Ajax と名付けられて、その後多くの Web アプリケーションに革命を起こしていった」

「ふむふむ」

「ただ当時、Ajax を活用して Web アプリケーションを開発するのはかなり難易度が高かった。XMLHttpRequest の実装はもちろん、DOM 操作やイベントハンドリングにまつわる仕様が各社ブラウザでバラバラだったため。そこで使われるようになったのが Prototype JavaScript Framework、通称 `prototype.js`⁴⁴。Ajax フレームワークを始めとした、JavaScript アプリケーション開発のためのさまざまな機能を統一した API で提供するもので 2005 年 2 月、奇しくも Google マップとほぼ同時に公開されたもの。Ruby on Rails において Ajax サポートの役割を担うものとして開発されたもので、しばらくデフォルトの JavaScript ライブラリとして Rails にバンドルされていた」

⁴⁴ <http://prototypejs.org/>

「へー、Rails とそんな因縁があったとは」

「そして翌 2006 年 8 月に jQuery⁴⁵ がリリース。JS アプリ開発のためのユーティリティライブラリ群なのは先行の prototype.js と同じながら、DOM 操作の API が非常にシンプルかつ直感的だった。

`$('#id123').text()` みたいな書き方ができるやつね。他にも CSS セレクタを備え、かなり軽量で動作が速かったことから、先行する prototype.js をあっという間に追い抜いて広く普及していった。そして JavaScript ライブラリといえば jQuery といっても過言ではないほどの圧倒的地位を築いたの。Rails もその後デフォルトの JavaScript ライブラリを、2011 年リリースのバージョン 3.1 から jQuery に乗り換えた」

「へー、そんな経緯があったんですね。Rails 使ってたのに知らなかった」

「従来はサーバサイドで担っていた処理の一部を、ブラウザに任せることによってユーザーの操作性やレスポンスを改善する技術を俗に『モダン Web』と呼ぶことがあるけど、そのモダン Web 開発のためのライブラリやフレームワークを世代分けするなら、prototype.js や jQuery はその第 1 世代に区分できる。他にも当時 Dojo Toolkit⁴⁶ や MooTools⁴⁷、YUI (Yahoo! User Interface Library) ⁴⁸ などがリリースされたけど、これらに共通するのは Ajax や DOM 操作を統一したインターフェースで使いやすくするためのツールキットだったということ。それ単体で複雑なアプリケーションを作るには向いておらず、そのためには Ruby on Rails などのサーバサイドフレームワークを組み合わせる必要があった」

6-2. 第 2 世代：SPA フレームワークの登場

「jQuery に代表されるモダン Web 第 1 世代のライブラリが普及し、大規模なアプリケーションにも採用されるようになると、いくつかの課題が浮き彫りになってきた。たとえば jQuery では任意の DOM 操作はイベントハンドラ⁴⁹に設定されることがほとんどだったんだけど、アプリケーションの規模が大きくなってくると複数のイベントハンドラが各自それぞれの思惑で任意の DOM 要素に変更をかける。するとユーザーが UI を色々操作した結果、どの DOM 要素がどのイベントハンドラによ

⁴⁵ <https://jquery.com/>

⁴⁶ <https://dojotoolkit.org/>

⁴⁷ <https://mootools.net/>

⁴⁸ <https://clarle.github.io/yui3/>

⁴⁹ Web アプリケーションにおけるイベントとは、マウスクリックやキー入力といったユーザー操作やページ読み込み完了などのシステム動作のこと。JavaScript には DOM 要素に対して特定のイベントが発生したときに任意の関数やメソッドが実行されるよう関連づけるしくみがあり、それをイベントハンドラ (Event Handler) という。

って変更され、現状がどうなってるのか把握しきれず開発者の想定を超えた事態が多発するようになる」

「ふむふむ、チームの誰かが入れた処理を全員が把握してないとバグの温床になりそうですね」

「jQuery の中だけでも大変なのに、これを Ruby on Rails などと組み合わせるとさらに厄介な事態が生じる。サーバサイドで稼働する Rails の Model と Controller の処理が View に出力され、その結果の HTML がブラウザに配信されるわけだけれども、その HTML はブラウザで動く jQuery 側から随時変更される。データフローがサーバサイドとクライアントサイドの双方からあり、互いにそれぞれが何をしているか検知する術がないため、UI が今どうなっているか誰にも責任が取れない」

「不具合が起こったら、まずサーバとクライアントのどちらに問題があるかの特定に手間取り、クライアントに問題があったらあったで、現象の再現に手間取るわけですか。複雑すぎて開発者にとっては悪夢でしょうね」

「当時はスマートフォンが普及し始め、モバイルアプリが次々作られるようになってきた時代。それに対応して、Rails にも RESTful API⁵⁰ を簡単に作成できる機能が実装されていた。だから Web アプリケーションもモバイルアプリのように処理の主体をクライアントのブラウザに移し、必要なデータは随時サーバ API から受け取ればいいんじゃないかと先進的な開発者たちが考えるようになり、SPA (Single-Page Application) のコンセプトが生まれたの」

「ちなみになんで『シングルページ』っていうんですか？」

「サーバからは HTML のページは初回のアクセスしか返さず、それ以降のユーザー操作はクライアントに配信されたプログラムがさばいて処理するからだね」

「えっ、でも今の X とか Instagram ってリンクをクリックしたときアドレスバーのパスが変わりますよね。/home から /notifications みたいに」

「あれはネットワーク上でページ遷移してるように見えるけど、実際はブラウザ内だけの処理で完結してるの。そのあたりは後でルーティングについて学ぶときにくわしく説明する」

「わかりました。それでそういうアプリケーションを SPA というなら、従来の Rails とかで作られる複数のページ構成のアプリケーションはなんて呼べばいいんですか？」

「そっちには MPA (Multi-Page Application) という用語が充てられるようになった。新しい技術が出てきたときに従来の範囲を特定の表すために作られる言葉のことを『レトロニム』っていうんだけど、MPA もその類だね。携帯電話に対する固定電話とか、粉ミルクに対する液体ミルクみたいな」

「モバイルスーツに対するノーマルスーツもそうですね！」

「そうして 2010 年前後には SPA の開発を想定したフレームワークがいくつもリリースされた。代表

⁵⁰ 異なるシステム間でインターネットを介してデータをやり取りする API の一種。データや機能が一意の URI で表現され、GET や POST、PUT、DELETE といった HTTP メソッドがセマンティックに用いられることが特徴。

的なのは AngularJS⁵¹、KnockoutJS⁵²、Backbone.js⁵³、Ember.js⁵⁴ あたりだね。これらに共通するのは、サーバサイドで一般的だった MVC パターンもしくはそれを独自にアレンジして踏襲していたこと。そして軽量の Backbone.js 以外は実際の DOM を直接操作するのではなく、テンプレート形式の View から HTML を出力するようになっていたこと。さらにサーバ API との通信をそれまでの XML ではなく、JavaScript で取り回しやすい JSON 形式で行うためのしくみを自前で用意していたこと」

「たしかに第1世代の jQuery とかとは考え方も作りも別物ですね。MVC パターンが使われているなら、私にもとつきやすそう」

「SPA 黎明期でまだ手探りの状況だったから、まずはサーバサイドで一般的だったパターンを導入したんだろうね。ただそっくりそのまま適用するのは難しかったのか、AngularJS は『MVW (Model-View-Whatever)』、KnockoutJS は『MVVM (Model-View-ViewModel)』、Backbone.js は『MVP (Model-View-Presenter)』パターンなどと称してた」

「どれも MVC から C の Controller が抜け落ちてますね。どうしてですか？」

「サーバサイドアプリではリクエストが来たら Controller が主体となって Model を操作し、処理結果を View にして出力すればそれで済む。対称的に SPA は初回のリクエストが終わってからが本番で、View に仕込まれたイベントハンドラを起点としてさまざまな処理が発生する。だから従来型の Controller が View を一方的に支配するアーキテクチャをそのまま適用するのが難しかったんだと思う」

「なるほど」

「サーバサイドアプリの View ではデータ値は埋め込んだら終わりだけど、SPA ではそうはいかない。オリジナルのデータの変更に追従して随時変更する必要がある。たとえば Excel で A3 セルに A1 と A2 セルの値の合計値を算出する $fx = SUM(A1:A2)$ という数式が適用されていたとする。すると A1 や A2 の値をいじるとそれに付随して A3 の値が即座に変わるよね。こういった UI がデータの変更に自動的に反応し、その変更を反映する機能性のことをリアクティビティ (Reactivity) と呼ぶの。以降の SPA フレームワークにおいて、このリアクティビティをいかに高パフォーマンスで、かつ使いやすい API で実現するかが最大の課題になるわけ」

「へーえ」

「これはソフトウェア工学では昔から研究されているテーマでもあって、それを実装するためのデザインパターンでもっともメジャーなのが Observer パターンね。別名 Publish-Subscribe モデルとも言って、出版社とその購読者の関係に喩えられる。完全に不定期発行の雑誌があったとして、そのファンはできる限り早くその最新号が読みたい。でもいつ発行されるかわからないから毎日書店に通って店

⁵¹ <https://angularjs.org/>

⁵² <https://knockoutjs.com/>

⁵³ <https://backbonejs.org/>

⁵⁴ <https://emberjs.com/>

頭チェックしてるんだけど、その場合ほとんどの訪問は無駄足になってしまうよね」

「ええー、最新号が発売されたら LINE とかメールで通知してくれるサービスがあったらいいのに」

「Observer パターンはまさにそれ。Subscription（購読）を登録しておく、Publisher に変更があれば自動的に通知してくれる。これをテンプレートのデータバインディングに使ったのが KnockoutJS。View と Model の間にバインディングを担当する ViewModel を定義し、そのプロパティを Observable として定義することでバインディングを実現した。サンプルを示すと、まず定義はこんな感じ」

```
var UserVM = {
  userName: ko.observable('Alice'),
  userAge: ko.observable(20)
};
ko.applyBindings(UserVM, document.getElementById('user'));
```

「この ViewModel のプロパティ値をテンプレートで使うにはこんなふうを書く」

```
<div id="user">
  Name: <span data-bind="text: userName"></span>
  Age: <span data-bind="value: userAge"></span>
</div>
```

「Observable は関数として呼び出されることで自身の変更を検知する。上の例で Alice ちゃんを 18 歳にするには UserVM.userAge(18) のようにコールするわけ。このやり方を KnockoutJS では宣言的バインディングと称したけど、書き方がまどろっこしいからか当時の開発者からはあまり人気がなかった。ただこの MVVM パターンは後に Vue.js も採用してるし、Ember.js は Observable を使って Model を直接 View にバインディングするアプローチを取った。KnockoutJS が後世に大きな影響を残したフレームワークであることはまちがいないね」

「へー、Vue にも影響を与えたフレームワークだったんですね」

「いっぽう、AngularJS はもっとシンプルに書ける方法でバインディングを実現した。Controller と View の間に \$scope というオブジェクトを介在させ、そこにバインディングしたいデータを片っぱしから登録する」

```
var app = angular.module('myApp', []);
app.controller('myController', function($scope) {
  $scope.user = {
    name: 'Alice',
    age: 20
  };
});
```

それをテンプレート側から参照するには次のように書く。

```
<body ng-app="myApp">
  <div ng-controller="myController">
    Name: {{ user.name }}
    Age: {{ user.age }}
  </div>
</body>
```

「年齢を変更したいときは `$scope.user.age` を直接書き換えればいい。またバインディングは基本的に双方向なので、フォームに設定すればそこで書き換えた値は元のデータに反映される」

「おおっ、KnockoutJS よりだいぶんわかりやすいですね！」

「AngularJS は Google のエンジニアだった Miško Hevery が同社の 20% ルールを利用したサイドプロジェクトとして始め 2009 年に公開したもので、その動機は大規模な SPA をより簡単に構築できるようにすることだった。すぐに開発者たちの話題になり、一番人気のフレームワークの地位に上り詰めた」

「ふむふむ。それにしても AngularJS は、明示的に記述せずにどうやってバインディングを実現してるんですか？」

「AngularJS は実行時にテンプレートをコンパイルするんだけど、そのときに `{{ }}` 記法で埋め込まれている変数を `$scope` で監視するよう登録する。イベントが発生したりページルートを移動したタイミングで `$scope` に登録されているすべての変数について前回の値と今回の値を比較し、差分があれば DOM を書き換えるというしくみになってるの。泥臭いやり方だから **Dirty-Checking** と呼ばれてる」

「何かあるたびに全部の値をチェックし直すんですか。たしかにスマートなやり方じゃなさそう」

「そして実際のアプリケーション開発では `$scope` を入れ子にすることが多く、バインドされた変数や式もしばしば依存関係を持っている。するとすべてのバインドした値が定まるまで `$scope` ツリーの頭まで巻き戻して何度も何度もループを繰り返す事態が生じる。ゆえにバインド値が増えるほど顕著にパフォーマンスが落ちてしまうため、公式はウォッチャに登録する値は 1 ページ 2,000 個までに留めるように推奨していた。けど本格的な SPA だと万単位のバインドが必要になることもあるので、大規模開発に対して AngularJS はあきらかにパフォーマンスの問題を抱えていたの。なおこの Dirty-Checking は Ember.js も採用していたので、こちらも同様の問題があった」

「なるほど。書き方が厳密だと開発者の人気がないし、書き方を簡単にしようするとパフォーマンスに問題が生じると。第 2 世代の技術にはそういうジレンマがあったんですね」

「そう。そういう状況のなかで、思わぬ方向から解決策を示したのが React だったわけ」

第7章 ビルドツールでプロジェクトを運用する

7-1. React 開発におけるビルドツールの変遷

7-1-1. React の開発環境を作るのに必要なツール

「初学者が React プロジェクトをどうやって始めるかは以前は悩むようなことではなかったんだけど、今はこれが非常に複雑な問題になってる。React の公式サイトが react.dev ドメインで 2023 年 3 月にリニューアルされた当初、プロジェクトの構築には Next.js や Remix のようなメタフレームワークを使う方法のみが紹介されてたの⁶⁹。しかしこれには私も含めて多くの開発者が違和感を抱いていたの」

「どうしてですか？」

「React が圧倒的なシェアを占めるようになったことで、以前のような技術に明るい開発者ばかりでなく趣味レベルのユーザーが増えた。実際に動くアプリを手っ取り早く作りたいという彼らのニーズに応えた構成だったんだろうけど、仕事で使う人にとっては最初から React を包含したメタフレームワークを使うのは理解をあやふやにしまうリスクをはらむ」

「たしかに、最初から Rails でアプリを作り始めて Ruby についての理解があやふやなプログラマは少なくないですね。そういう人が作ったアプリのケアは絶対したくない（笑）」

「フレームワークにはそれ独自の流儀や都合がある。Next.js から始めるとどこまでが React の仕様でどこからが Next.js の仕様なのかわからず、いざ問題が起きたときに行き詰まりやすい。プロなら何が起きても自分で対応できるように、基礎からちゃんとその原理を理解しておくべき。だからまずは React 単体で動く SPA の環境で、React の基本を学ぶでほしいの」

「そうですね。私もそう思います」

「やっぱりそういう声が多かったせいか、公式ドキュメントに 2025 年 2 月に『ゼロからの React アプリ構築』⁷⁰というページが追加された。これはビルドツールを使って React を SPA としてセットアップする方法を説明したもの。私たちはこちらを採用しよう」

「公式ドキュメントでは環境のセットアップに Vite、Parcel、Rsbuild という 3 つのツールによる方法が紹介されてますね。そういえば最初に『Hello, World!』のプロジェクトを Vite で作りましたっけ⁷¹。この Vite が React プロジェクトを SPA としてセットアップしてくれるツールなんですか？」

⁶⁹ 「React アプリの作成 - React」 <https://ja.react.dev/learn/creating-a-react-app>

⁷⁰ 「ゼロからの React アプリ構築 - React」
<https://ja.react.dev/learn/build-a-react-app-from-scratch>

⁷¹ 【① 言語・環境編】の「1-2. Vite でプロジェクトを作成する」を参照。

「`pnpm create vite` の実行は `pnpm dlx create-vite` へのエイリアスになってる。create-vite はビルドツールである Vite を基盤にプロジェクトのひな形生成をしてくれる公式のパッケージコマンドだね」

「ビルドツールってひな形生成をしてくれるものじゃないんですか？ 用語のニュアンスがわからなくて混乱してきました……」

「Ruby on Rails はセットアップに必要なものが最初からそろっていて、これに該当する概念がないからね。JavaScript / TypeScript の世界では開発環境にまつわるツールが豊富に出回っていて、かつ最近では既存のカテゴリーに収まらないプロダクトも多く説明するのが難しいの。

アプリケーションのビルドには複数の工程が必要で、ビルドツールは特定の工程を担わせるために内部に他のツールを組み込んでいることも多い。そして各種の処理を自動化・効率化するツール群を連携して統合的に開発プロセスをサポートしてくれるプロダクトを**ツールチェーン**という。Vite はビルドツールであり、かつそれ以上に包括的な機能を提供してくれるツールチェーンでもある」

「うーむ、まだ混乱してます……。Vite というツールを理解するには、そのひとつひとつの機能を解きほぐして考えないといけないというわけでしょうか」

「そうだね。じゃああらためてプロジェクトを生成する前に、なぜ今の React 開発で Vite が広く使われるようになったのか、ひいては create-vite コマンドは何をしてくれているのかということを学んでおこうか。どっちにしる最終的には自分で環境を作って本番運用できるようになってもらわないといけないので、順番的には今知っておいてもらって損はないし」

「そうですね、ぜひ教えてください！」

「そのためにまずどういう経緯でどんなカテゴリーのツールが生まれたのか、そしてそれらが廃れたり統合されていったのかということを、やはり歴史を振り返って説明する必要がある。2013 年のバージョン 0.3 公開当初、React を使うにはソースコードファイルを CDN⁷² から `<script>` タグで読み込むという原始的な手法が用いられていた。JSX も公式が提供する `JSXTransformer.js` というライブラリを読み込んで、ブラウザ実行時に JavaScript に変換してたの」

「へー、React にそんな泥臭い環境で開発してた時代があったんですね」

「最初の転換期は ES2015 (ES6) の登場。このおかげで JavaScript が一気にモダンな言語になったという話はしたよね⁷³。これを当時のブラウザでもいち早く使いたい開発者のために作られたのが

6to5。その名のとおり ES6 のコードを ES5 にトランスパイルしてくれるツールだった」

「そのトランスパイルというのは？ コンパイルとはちがうんですか？」

「コンパイルとは、狭義にはある言語で書かれたコードをより低水準の言語に変換するプロセスのこと。いっぽうトランスパイルは同水準の別の言語もしくは同じ言語の異なるバージョンに変換するプ

⁷² Content Delivery Network の略で、インターネットでコンテンツを効率的かつスピーディに大量配信するためのしくみ。コンテンツを地理的に分散したサーバグループにキャッシュし、エンドユーザーにもっとも近い経路にあるサーバから配信する。

⁷³ 【① 言語・環境編】の「2-1-2. 年々進化していく JavaScript」を参照。

ロセスを指すの。広い意味ではトランスパイルはコンパイルの一種とされることも多い。2014年にリリースされた6to5はほどなくJSXのサポートを追加し、プロダクト名もBabel⁷⁴に変更された。このBabelやtscといったコンパイラが当時から今に至るまでモダンWeb開発に必須のツールになっている」

「ふむふむ」

「またそのころ、すべてをスクリプトとしてフラットにブラウザに読み込ませるのではなく、JavaScriptレベルでモジュールとして扱いたいというニーズが高まっていた。Node.jsの環境でrequire()を使って他のJavaScriptファイルを読み込むCommonJSというモジュールシステムが有志によって作られた経緯は話したよね⁷⁵。CommonJSを使って書いたコードをブラウザで動くようにするには、その依存関係を解析してrequire()文がないひとつのJavaScriptファイルにまとめてやる(=バンドルする)必要がある。そのツールのことをモジュールバンドラといい、React開発において最初に人気を集めたモジュールバンドラがBrowserify⁷⁶だった」

「公式サイトを見るとBrowserifyのロゴ、魔法使いの帽子がモチーフなんですね」

「<script>タグによって手動で管理していた状況に比べると、どのファイルがどの順番で読み込まれるかを気にせずrequire()で必要なモジュールを宣言するだけでよくなったこと、npmパッケージがそのままフロントエンドで使えるようになったのは、まさに魔法のように画期的なことだったんだと思うよ」

「なるほどー」

「当時のビルドというのは、主にこのコンパイルとバンドルのことを指していた。そしてコンパイラやモジュールバンドラの実行を任意のワークフローとして自動化するため、Grunt⁷⁷やgulp⁷⁸といったタスクランナーが利用されてた。というわけで以上がReactが登場してから数年間の状況だった」

7-1-2. webpack 帝国の完成

「CommonJSはJavaScript開発によりやく実用的なモジュールシステムをもたらしてくれたものだったけど、読み込みが同期的にしか行われない、構文が柔軟すぎるために静的解析が困難で出力ファイルの最適化が難しいという問題を抱えてた。そこで後からAMD(Asynchronous Module Definition)

⁷⁴ <https://babeljs.io/>

⁷⁵ 【① 言語・環境編】の「2-9-1. JavaScript モジュール三國志」を参照。

⁷⁶ <http://browserify.org/>

⁷⁷ <https://gruntjs.com/>

⁷⁸ <https://gulpjs.com/>

が生まれ、さらに ES2015 に ES Modules が公式な仕様として盛り込まれたんだよね。将来的には ESM に統一されるだろうことは予測できたけど現実として CommonJS による膨大な資産が存在し、作者の好みから AMD 形式で提供されているライブラリも少なからずあった。この三者が混在する状況の中、救世主となったのが **webpack**⁷⁹ だった」

「webpack！　どういうものかはわかってませんが、その名前はよく聞きます」

「webpack は複数のモジュール形式をネイティブサポートするバンドラで、当初から CommonJS と AMD に対応しており、2016 年リリースのバージョン 2 から ESM にも対応した。その後バージョンを重ねてモジュール形式を自動判別し透過的に扱えるようになり、ユーザーがモジュール形式のちがいを全く気にせずコードが書けるようになったの」

「React ユーザーにとって、具体的にはどういうメリットがあったんでしょう？」

「まだ普及初期にあった ESM 形式に書き方を統一できたことだね。当時は React を含めメジャーなパッケージの多くが CommonJS 形式で提供されていたけど、webpack を使えば『そんなの関係ねえ！』とすべて `import` 文で読み込めたわけ。

そして webpack のもうひとつの特徴は高い拡張性。デフォルトでも JavaScript に加え JSON や画像などのアセットファイルもモジュールとしてバンドルできたんだけど、他の種類のファイルに広く対応するためにローダー⁸⁰ というしくみが用意されてる。これを利用すれば ES2015 以降や JSX、TypeScript、CSS およびその拡張言語も webpack の環境で透過的に記述できるようになる」

「ふーむ。私たちが今は普通に享受できていることばかりですけど、webpack が率先して始めてくれたことだったんですね」

「そして webpack はただのモジュールバンドラにとどまらず、コアおよび標準プラグインによって以下のような機能も提供してる」

- **Minification** 空白文字やコメント、実行されないコードやコンソール出力などを削除、変数名や関数名を短縮、条件文や宣言文を簡略化するなどして、スクリプトファイルの容量を最小限に圧縮する
- **Tree Shaking** どこからも参照されていないモジュールを検出してバンドル対象から除外し、出力ファイルの容量を削減する
- **Code Splitting (コード分割)** モジュールの共有度合による自動分割に加え、開発者がアプリケーションの UI や規模を考慮しつつ、バンドルファイルを適切な粒度に分割し、それぞれが適切なタイミングで読み込まれるようにできる

⁷⁹ <https://webpack.js.org/>

⁸⁰ <https://webpack.js.org/loaders/>

https://developer.mozilla.org/ja/docs/Web/HTTP/Basics_of_HTTP/Data_URLs

- **キャッシュ管理** ビルド時に出力ファイルの名前に差分単位でユニークなハッシュ値⁸¹を付与することで、ブラウザキャッシュが適切に使われるようにし、かつキャッシュが残ることによる挙動の不具合を予防する

「出力ファイルのパフォーマンス最適化まで面倒見てくれるようになったんですね。たしかにこれはモジュールバンドラというカテゴリーに収まらないプロダクトだ」

「さらに webpack の普及を後押ししたのが webpack-dev-server⁸² の存在。それまでは開発中のアプリケーションの実行結果を確認するために、いちいち全体をビルドし直して出力ファイルをブラウザで再読み込みするのが一般的だった。webpack-dev-server は webpack 環境で動作する開発サーバで、メモリ上での高速な再ビルドやブラウザの自動リロード、後にはアプリケーションの状態を維持したまま変更されたモジュールだけを置き換える HMR (Hot Module Replacement) という機能も備えるようになり、ユーザーの開発体験を大きく向上させた」

「それも webpack が最初だったんですね。いいことばかりのように思えますけど、webpack には欠点はなかったんでしょうか？」

「webpack は高い柔軟性と多くの機能を備えていたわけだけど、その恩恵を享受するために必要な設定が恐ろしく複雑だった。ローダーやプラグインといった抽象的な概念を理解したうえで、それらを組み合わせて自分の環境に適したビルドパイプラインを構築することが求められる。しかし初期のドキュメントは断片的でわかりづらく、相性の悪い組み合わせで予期せぬエラーに悩まされることも多かった。コミュニティにはその設定はまるで黒魔術の呪文のようだとか、チームに専門の webpack 職人が必要だといった声もあったほど」

「webpack 職人の朝は早い——。……すいません、言ってみただけです」

「この状況は React が成長曲線に乗り始めていた当時の Facebook にとって憂慮すべきものだった。熟練の webpack 職人がいないと、実用レベルの React の開発環境が作れないわけだからね。そこでコアチームのメンバーだった Dan Abramov が中心となり、社内で使われていた原型的なツールを一般向けにブラッシュアップして OSS として公開することを決めた。それが **Create React App (CRA)**⁸³ で、2016 年 7 月にバージョン 1.0 がリリースされると瞬く間に React コミュニティにおける標準ツールになった」

「create-vite とか create-next-app と似たような名前のツールは多いですね。これがその走りなんですか？」

「そう。あまりにも成功したため、npm が `npm init <initializer>` が内部で `npx create-`

⁸¹ データを一意に識別する固定長の数値のこと。大量のデータをはるかに小さい数値として表すため、ファイルの改ざん検知や同一性の確認に役立つ。

⁸² <https://webpack.js.org/configuration/dev-server/>

⁸³ <https://create-react-app.dev/>

`<initializer>` を呼び出す仕様に^な変更されたほど。だから後発のツールもこの形式の名前に倣^なっているというわけ」

「webpack の環境構築が難しいから CRA が作られたということでしたけど、CRA と webpack はどういう関係にあるんですか？」

「create-react-app コマンドで生成したプロジェクトには webpack や webpack-dev-server、Babel、それに静的コード解析ツールの ESLint、テストフレームワークの Jest といった当時の React 環境におけるベストプラクティスな組み合わせのツール群がインストールされてる。それらは `react-scripts` というスクリプトコマンドを通して使うようになっていて、ユーザーからはできるかぎり隠蔽されてる。生成される `package.json` の `scripts` プロパティを見てみよう」

```
"scripts": {
  "start": "react-scripts start",
  "build": "react-scripts build",
  "test": "react-scripts test",
  "eject": "react-scripts eject"
},
```

「`react-scripts` は Node.js の環境で実行されるスクリプトで、そこを起点にサブコマンドが `start` なら webpack-dev-server を起動し、`build` なら webpack でバンドルを実行するといった具合に各ツールにタスクが割り振られるの。CRA はビルドツールよりコードを書くことに集中してもらいたいという哲学のもとに作られてるので、ユーザーは webpack の設定などを自分でカスタマイズすることは一切できない」

「えっ。でも商業レベルの大規模なアプリケーションとかだと、お膳立てされた環境では間に合わないシーンも出てくるでしょう？」

「だから `react-scripts` には `eject` というサブコマンドがあって、それを実行すると `react-scripts` パッケージがアンインストールされ、隠蔽されていたツールを直接使うようプロジェクトが再構築される。具体的には `package.json` が書き換えられて `react-scripts` の下に隠れていた 50 近くのパッケージが `dependencies` のリストに追加され、`webpack.config.js` などの各種設定ファイルが作られる。不可逆なので慎重に行う必要があるけど、これにより最初はプロジェクトを CRA で作成し、規模が大きくなって高度なカスタマイズが求められるようになったらその時点で `eject` するといった柔軟な運用が可能になるわけ」

「なるほど。そういうところも人気の要因だったんですね」

第 8 章 コンポーネントの基本を学ぶ

8-1. UI コンポーネントライブラリの導入

8-1-1. UI コンポーネントライブラリとは

「React でコンポーネントを作成していくにあたって、あるていど見た目をよくするために **UI コンポーネントライブラリ**を導入しておきたい。統一された外観を持つ UI 要素をコンポーネントの形で提供してくれているライブラリのことね。本格的な大規模プロジェクトならひとつずつ CSS で記述していくところだけど、既存の UI コンポーネントライブラリを使えばそれなりに見た目の整ったアプリケーションを短時間で構築できるのね」

「Bootstrap¹²³ や Foundation¹²⁴ のような CSS フレームワークは使ったことがあるんですが、それとはまたちがうんですか？」

「UI 要素が React のコンポーネントの形で提供されていて、見た目のみにとどまらず機能までが統合されてるの。だからモーダルだったり一定時間で消える通知なんかがコンポーネントの形で簡単に実装できる。React コミュニティでよく使われているライブラリだと Material UI (MUI)¹²⁵ や Ant Design¹²⁶、Chakra-UI¹²⁷、Mantine¹²⁸ あたりをよく聞くね」

「その中で一番使われてるのは？」

「MUI だろうね。Google の Material Design ガイドラインに準拠しててモダン、提供するコンポーネントの種類が多い、カスタマイズ性が高いというのが人気の理由。Spotify や NASA、Netflix といった大手企業の採用実績もあるそうだよ」

「そんな有名企業も UI コンポーネントライブラリを使ってるんですね。じゃあやっぱりウチでもその MUI を使うんですか？」

「少し前までなら Yes と答えてたんだけど近年、UI コンポーネントライブラリの変える新星が出現してすごい勢いで支持を集めてるの。shadcn/ui¹²⁹ というプロダクトで、リリースされた 2023 年だけで約 4 万の GitHub スターを獲得。翌 2024 年もその勢いは衰えず、全 JavaScript プロジェクトに

¹²³ <https://getbootstrap.com/>

¹²⁴ <https://get.foundation/>

¹²⁵ <https://mui.com/>

¹²⁶ <https://ant.design/>

¹²⁷ <https://chakra-ui.com/>

¹²⁸ <https://mantine.dev/>

¹²⁹ <https://ui.shadcn.com/>

「あ、ボタンが shadcn/ui の黒いボタンになりましたね」

「デフォルトの CSS 設定と Tailwind CSS のスタイルがうまく噛み合わなくてところどころ崩れてるけど、不要な CSS は順次削除していくね」

8-2. コンポーネントに Props を受け渡す

「お待たせしました！ ここからついに React コンポーネントの基本について学んでいくよ。前提の確認だけど、React は機能をカプセル化したコンポーネントをベースとしたアーキテクチャが採用されていて、コンポーネントをツリー上に組み合わせることでアプリケーションを構築するようになってるというのはすでに理解してるよね。そして UI のリアクティビティを実現するために、コンポーネントツリーを仮想 DOM としてメモリ上に持っていて、そのために活用していることも」

「はい。丁寧に説明してもらったので、わかってるつもりです」

「そのしくみゆえに React におけるアプリケーション開発では、コンポーネントの再レンダリングが大きな関心事になる。インタラクティブに UI を書き換えるためには、該当するコンポーネントを再レンダリングする必要があるからね。React では任意のコンポーネントが再レンダリングされるのは、次の 3 通りのいずれかに該当したときになる」

1. コンポーネントが呼ばれる際のプロパティ値 (**props**) が更新された場合
2. コンポーネント自身の状態 (**state**) が更新された場合
3. 親コンポーネントが再レンダリングされた場合

「3 は Reconciliation プロセスの中で従属的に起きるものだから、コーディング時にあえて意識することは少ない。だから主に 1 と 2 のケースを考慮しながらコーディングすることになる」

「ふむふむ」

「コンポーネントを設計するときに意識するべきはこの props と state。じゃあそのどちらを最初に考えるべきかというと、断然 props なの。なぜなら関数型プログラミングのアプローチによって宣言的 UI を実現しようとする React においては、状態はコンポーネントに極力持たせるべきではないものだから。state を持つコンポーネントの数は最小限に抑えるべきだし、ひとつのコンポーネントが持つ state の数も最小限にするべき」

「どうしてですか？」

「React においてコンポーネントを関数として考えるというのは、単なるメタファーにとどまらない。もしアプリケーションの構成単位であるコンポーネントを純粋関数ですべて表現できれば、アプリケーション全体における副作用を完全に排除できる。ただ現実問題として状態を完全に持たないア

アプリケーションというのはいりえないので、設計段階でそれを最小限にしたうえで、かつその影響範囲を限定させたいわけ」

「なるほど」

「純粋関数とは、引数が同じなら必ず同じ戻り値を返す関数のこと。ゆえに React にとって理想的なコンポーネントとは、props が同じなら必ずレンダリング結果が同じになるコンポーネントなのね。コンポーネントにとって props がもっとも重要というのはこういった意味を含んでる。だからまず先に props について学んでいこう」

「わかりました」

「props については JSX の説明のときに概要は話したよね¹⁵⁰。でも大事なのでおさらいしておこう。コンポーネントを関数として考えたとき、その引数に相当するもの。JSX にマウントする際のタグの中では、その属性値として記述される。React が広めた概念で、同じコンポーネントベースのアーキテクチャを採用しているフレームワークでも相当するものを同様に props と呼ぶ。名前の由来は

『properties』から」

「はい、そうでしたね」

「そして仮想 DOM を形成する React Elements、具体的には `ReactElement` オブジェクトにおいては props プロパティに { 属性名: 属性値 } の形式のオブジェクトとして格納される。その React Elements から該当するコンポーネント関数が実行される際に、props オブジェクトを第1引数として受け取る。

説明としては以上だけど、具体的なコードを見ないとピンとこないだろうから、ここからは実際のアプリケーションを用いて学んでいこう。まずは用意してあるサンプルを実行してみよう¹⁵¹」



¹⁵⁰ 「5-2-2. JSX とコンポーネントの関係」を参照のこと。

¹⁵¹ <https://github.com/kleimiary/Riakuto-StartingReact-ja5.0/tree/main/08-basics/02-props>

図 20: SLAM DUNK 登場人物リスト

「あ、スラダンだ！ 柴崎さん、ちょいちょいこういうネタはさんできますよね」

「無味乾燥なサンプルより、こういうほうがやる気になるでしょ」

「そうですね。ちなみに柴崎さんはこの中で誰が一番好きですか？ 私はミッチー！」

「.....はいはい、そういった話は休憩時間にね。いまは React のお勉強。まずは App.tsx を見てみよう」

リスト33: 02-props/src/App.tsx

```
import PlayerList from '@components/PlayerList.tsx';

const title = import.meta.env.VITE_APP_TITLE;
const players = [
  {
    id: 1,
    name: '桜木花道',
    grade: 1,
    height: 189.2,
  },
  {
    id: 2,
    name: '流川 楓',
    grade: 1,
    height: 187,
  },
  {
    id: 3,
    name: '宮城リョータ',
    grade: 2,
    height: 168,
  },
  {
    id: 4,
    name: '三井 寿',
    grade: 3,
  },
  {
    id: 5,
    name: '赤木剛憲',
    grade: 3,
    height: 197,
  },
],
```

```
];

function App() {
  return (
    <main className="flex flex-col items-center justify-center min-h-screen space-y-8">
      <h1>{title}</h1>
      <PlayerList school="湘北高校" players={players} />
    </main>
  );
}

export default App;
```

「ちなみに見た目がリッチなのは shadcn/ui を使ってるからね。UI コンポーネントライブラリや Tailwind CSS についてはすでに説明済みなので、今回はそのへんについては省略するよ」

「えーっと、頭から確認させてください。選手のデータが格納された `players` というオブジェクト配列を作成し、インポートした `PlayerList` というコンポーネントに属性 `players` の値として設定してますね。あと `school` 属性にも `湘北高校` という文字列を設定してます」

「そう、それらの属性値がコンポーネントの実装側で `props` として受け渡されるわけね。じゃ次はこの `PlayerList.tsx` のコードを見てみようか」

リスト 34: 02-props/src/components/PlayerList.tsx

```
import { UserRound } from 'lucide-react';
import { Avatar, AvatarFallback } from '@components/ui/avatar.tsx';

interface Player {
  id: number;
  name: string;
  grade: number;
  height?: number;
}

interface PlayerListProps {
  school: string;
  players: Player[];
}

function PlayerList(props: PlayerListProps) {
  const { school, players } = props;

  return (
    <div className="w-96">
      <h2 className="mb-8 text-center">{school}</h2>
```

```

<ul className="my-6 space-y-5">
  {players.map((player) => (
    <li key={player.id} className="flex items-center space-x-4">
      <Avatar>
        <AvatarFallback className="bg-red-50">
          <UserRound className="text-red-500" />
        </AvatarFallback>
      </Avatar>
      <div className="ml-4 text-left">
        <p>{player.name}</p>
        <p className="flex text-gray-400">
          <span>{player.grade}年生</span>
          <span className="mx-2" />
          <span>{player.height ? player.height : '???'}cm</span>
        </p>
      </div>
    </li>
  ))}
</ul>
</div>
);
}

export default PlayerList;

```

「ふむふむ。まず個別の選手のデータを格納するオブジェクトの型定義をするための `Player` がインターフェースとして定義されていて、さらに Props インターフェースの `players` プロパティがその配列の定義に利用されてる。Props インターフェースはその下で `PlayerList` コンポーネントの `props` の型として適用されてますね」

「そのとおり。props の型を設定することで、そのコンポーネントを JSX で使うときに必要な属性値とその型に縛りが発生する。だから `App.tsx` で `<PlayerList>` をマウントするとき、`school` と `chracter` の属性値をそれぞれ適正な型で記述しないと怒られてしまう」

「ほんとだ。VS Code で `App.tsx` の `<PlayerList>` タグ内の `school="湘北高校"` の部分を削除してみたら、props が足りないって怒られました。ところでこっち側の変数 `players` では宣言時に `Player[]` を適用してないはずなのに、props として渡してもエラーにならないのはなぜですか？」

「一度説明したけど¹⁵²、TypeScript における型の整合性チェックは構造が同じなら同じ型と見なす**構造的サブタイピング**を採用してるからだね。ために `players` のプロパティ名とかをいじってみたらエラーになるはずだよ」

¹⁵² 【① 言語・環境編】の「4-5-4. 関数のオーバーロード」を参照のこと。

「たしかに `grade` プロパティを `year` に書き換えたりすると、`players` のところに赤の波下線が出て型エラーになりますね」

「うん、これからも疑問に思ったことはそうやって自分で色々試してみて、実際にエラーを出して確認するというのは積極的にやっていてね。 `PlayerList` 関数の実装のほうに話を戻そう。関数コンポーネントは第1引数として `props` オブジェクトを受け取るようになってる。 `props` オブジェクトは `props.school` や `props.players` のようにプロパティにアクセスして使う方法もあるけど、ここでは取り回しのいいように分割代入で `school` と `players` を抽出してる」

「なるほど」

「その下ではレンダリングしたい内容を `return` で返してる。関数コンポーネントでは戻り値として設定できるものは `React Elements` の他に、文字列や数値、`Boolean` 値、`null` がある。条件によって何も `UI` に出力させない場合は `null` を返すようにする」

「ここでは普通に `JSX` で記述したものを返してるので、戻り値は `React Elements` になるんですね。その中身を確認させてください。 `<Avatar>` と `<AvatarFallback>` は `shadcn/ui` で作成したコンポーネントですよ。それからリスト要素の中でオブジェクト配列の `players` を `map()` によって繰り返し処理してると.....。

あれ？ 2つめの `<p>` の中の `{player.height ? player.height : '???'}` のとこって何をやってるんですか？」

「`Player` オブジェクトの `height` プロパティは省略可能なので、三項演算で `undefined` だった場合は文字列 `???` を代わりに表示するようにしてるわけ。省略可能プロパティは便利だけど、省略されたときに `UI` をどうするかをちゃんと考えてあげないと、表示が変なことになるから気をつけてね」

「わかりました」

「それから見落としてほしくないのが、繰り返し要素である `<li key={player.id}...>` のところ。 `JSX` で要素をループ処理によって記述する場合、各要素にユニークな値を `key` 属性として設定しなければならない、という話をしたのをおぼえてる？¹⁵³」

「はい、そういう記憶はあります」

「今なら仮想 `DOM` の `Reconciliation` 機構の話をした後だから理解しやすいと思う。再レンダリングを効率化するため、並列になってる複数のコンポーネントの差分を検出するのに、何かしらコンポーネントを一意に特定する値が必要なのね。それが `key props` というわけ」

「なるほど」

「ここでは繰り返し要素の最上位にある `<li>` に `key props` として `id` プロパティ値を渡してあげてる。 `key props` をつけ忘れると、開発環境では `React` がブラウザのコンソールに `warning` を出して指摘してくれる。また静的コード解析用ツールを使って `React` 用の各種ルールを適用すると `VS Code` でコーデ

¹⁵³ 「5-2-3. `React` の組み込みコンポーネント」を参照のこと。

ィング中に指摘してくれるようになるんだけど、これはまた後でやり方を教えてあげる予定」

「配列になってるオブジェクトにユニークな ID がない場合は、key props にループのインデックスを使っていいんじゃないっけ？」

「仕方がない場合の最終手段としてならね。ただしその場合でも、リストが並べ替えられたりフィルタリングされることが絶対にない UI にしないといけない。理由はわかるでしょ？」

「あー。key 値でコンポーネントを特定してるのに、並べ替えやフィルタリングが起きると別のコンポーネントにその key 値が割り振られてしまって、再レンダリング時に表示が混乱させられるから？」

「そのとおり。だからリストになるデータのスキーマ設計時には、ユニークな ID をつけるのを絶対に忘れないこと。じゃ最後に、この `PlayerList` コンポーネントの記述を簡略化できる箇所を書き換えておこう」

リスト35: `PlayerList` コンポーネント簡略化の記述

```
- function PlayerList(props: Props) {
-   const { school, players } = props;
+ function PlayerList({ school, players }: Props) {
    return (
      :
      <p className="flex text-gray-400">
        <span>{player.grade}年生</span>
        <span className="mx-2" />
-       <span>{player.height ? player.height : '???'}cm</span>
+       <span>{player.height ?? '???'}cm</span>
      </p>
      :
    )
  }
}
```

「props は関数の中で移し替える代わりに、関数の定義時に分割代入で受け取れることもできるんですね」

「そう。シンプルなのでこれからはこの書き方にしよう」

「2 箇所めに書き換えてるところは、この書き方って何っていうんじゃないっけ？」

「これは Nullish Coalescing¹⁵⁴ だね。左辺が `null` か `undefined` の場合に評価が右項に渡される」

「そうそう。そのヌリッシュ・コアレスシングでした」

「三項演算子でも書けるけど、オプションなプロパティで値が省略されてるケースに適用するのであれば、Nullish Coalescing を使うほうが文脈的にも適切だね。文も短くなるし、使えるときは積極的に使いましょう。」

¹⁵⁴ 【① 言語・環境編】の「2-7-2. Nullish Coalescing と Optional Chaining」を参照のこと。

ここまでで props の定義と渡し方は理解できたよね。じゃあ渡される props の値が変わればそのコンポーネントが再レンダリングされるというのを実際に目で確認しようか。ブラウザ拡張として **React Developer Tools** という開発者向けのデバッグ用ツールが Meta から提供されているの¹⁵⁵。まずはこれをインストールしておこう」

「えーっと、Chrome 用の拡張をインストールっと。はい、終わりました」

「次にサンプルプロジェクトを実行しているタブで Chrome デベロッパーツールを開く。そしてツールペインの『 Components』というタブを選択して。タブの数が多いと隠れて見えないこともあるので、そういうときは右端の『>>』ドロップダウンボタンを押すと格納されてるメニューが表示されるよ」

「はい、Components タブを選択しました」

「最初に設定を確認しておこう。歯車アイコンから設定モーダルを開き、その『General』タブの『Highlight updates when components render.』を有効にする。これで再レンダリングがあるとそのコンポーネントの枠が一瞬光って教えてくれるようになる」

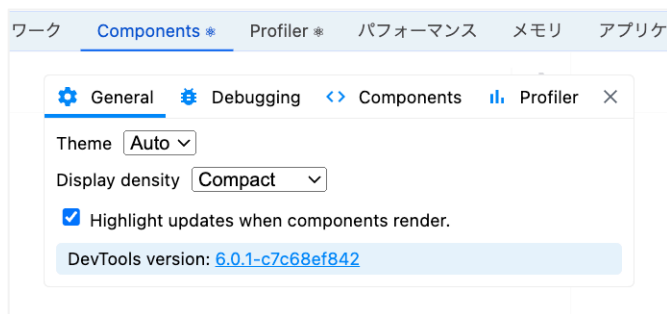


図 21: React Developer Tool 設定

「なおこのコンポーネントツリーは、デフォルトでは `div` や `li` のような組み込みコンポーネントはフィルタリングされてる。設定モーダルの『Components > Hide components where... > type equals dom nodes (e.g. <div>)』というフィルターを無効化すれば表示されるようになるよ。

それじゃツールから props を変更してみようか。パネル左側に表示されているコンポーネントツリーから『PlayerList』を見つけて選択。すると右側にそのコンポーネントの情報が表示されて渡されてる props が確認できる。この `school` の値 “湘北高校” を “神奈川県立湘北高校” に書き換えて Enter キーを押してみて」

¹⁵⁵ <https://ja.react.dev/learn/react-developer-tools>

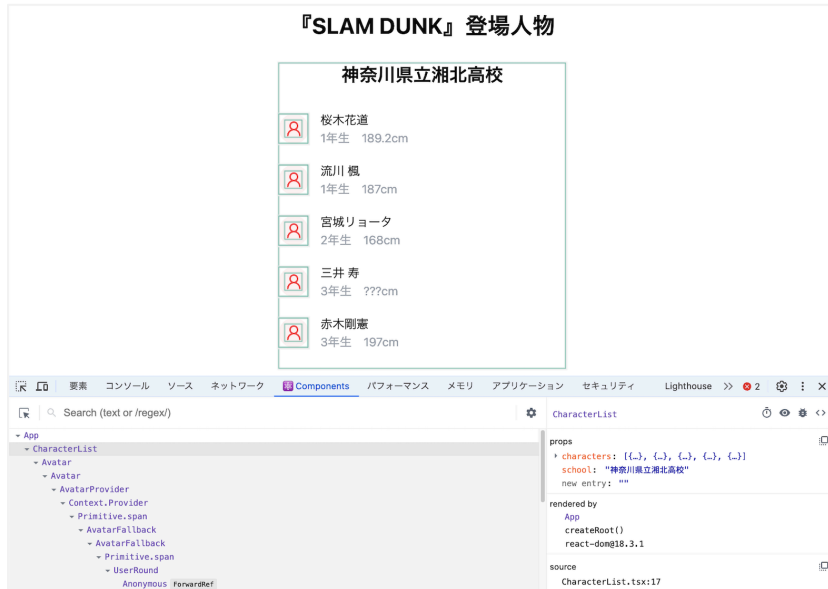


図 22: props を変更したところ

「あっ、高校名が変更されると同時に `PlayerList` コンポーネントとその子孫コンポーネント群の枠が一瞬光りました！ これが再レンダリングされたコンポーネントなんですね。でも props が更新された `PlayerList` はわかるんですけど、他のコンポーネントも再レンダリングされたのはなぜですか？」

「うん。まず高校名が表示される `Title` コンポーネントに関しては、`school` が子要素になっている。子要素は実は `children` という名前の props というのはすでに説明したよね¹⁵⁶」

「言われてみれば。いま思い出しました」

「加えて今回の冒頭で話した、コンポーネントが再レンダリングされる3つのケースを思い出して。3つめは『親コンポーネントが再レンダリングされた場合』だったでしょ。`Title` コンポーネントもそうだし、選手情報が表示されているコンポーネントもこれに合致する」

「なるほど。React ではあるコンポーネントが再レンダリングされると、その子々孫々のコンポーネントも道連れに再レンダリングされるんですね」

「そのとおり。じゃ次は同じようにツールから props の `players` を開いて4つめのミッチーのデータで `new entry` を `height` に書き換えて、値を `"` から `184` に設定して Enter キーを押してみてください」

「柴崎さんも『ミッチー』って言うんですね。はい、さっきと同様にコンポーネントの枠が光って三井先輩の身長データが更新されました」

「で、何か気づくことは？」

¹⁵⁶ 「5-2-2. JSX とコンポーネントの関係」を参照のこと。

「えっ？ うーん……。あ、そうか。React の Reconciliation 機構は値渡しのプリミティブ型の props の更新だけじゃなく、参照渡しオブジェクト型の props のプロパティが更新された場合も検知して再レンダリングしてくれるんですね？」

「そう。多重配列オブジェクトだろうが循環参照プロパティだろうが、中身に変更があれば React は検知して再レンダリングしてくれる。また、これについては次に話す state も同様のアルゴリズムが適用される。コンポーネントの props について知っておいてほしいことは以上ね」

8-3. コンポーネントに状態を持たせる

「次はコンポーネントの二大関心事のもういっぽう、状態について学んでいこう。コンポーネント関数内のローカル変数はレンダリングのたびに初期化されるため、UI の状態を保持するのには使えない。コンポーネントに状態を持たせるには **state 変数** というものを利用する。そして state 変数を作成するには `useState()` という組み込み API を使うの。その使い方はこんなふうになる」

```
const [count, setCount] = useState(0);
setCount(100);
```

「`useState()` は引数として state 変数の初期値を取り、戻り値として state 変数とその state を更新する **set 関数** をタプル¹⁵⁷にして返す。それらは一般的に、上記のように分割代入で受け取る。配列の分割代入と同様なので state 変数とその set 関数の名前は好きなものに設定できるけど、state 変数が `foo` だったら set 関数は `setFoo` にするのが慣例になってる。なお `useState` の TypeScript における型は次のように定義されてる」

リスト36: @types/react での `useState` の型定義

```
function useState<S>(initialState: S | (() => S)): [S, Dispatch<SetStateAction<S>>];
:
type SetStateAction<S> = S | ((prevState: S) => S);
type Dispatch<A> = (value: A) => void;
```

「`useState` コール時に型引数を指定しなかった場合、state 変数の型は引数として渡された初期値から推論した型が適用される。さっきの例では `0` を渡してるから `Number` 型になるわけね。単純な値なら

¹⁵⁷ 【① 言語・環境編】の「4-2-4. タプル型」を参照のこと。

それでいいんだけど、たとえば通常は文字列だけど初期値に `null` を取るような値を `state` にしたい場合などはそれではうまくいかない。だから次のように明示的に型引数を渡してやる必要があるの」

```
// 基本的に文字列だが、初期値をnullにしたい場合
const [id, setId] = useState<string | null>(null);

// 任意のオブジェクトの配列だが、初期値が空配列で型推論が効かない場合
const [players, setPlayers] = useState<Player[]>([]);
```

「なるほど、気をつけます」

「それじゃ実際に `useState` を使ったサンプルコードを見ていこう¹⁵⁸。まずは実行して動作を確かめてみて」

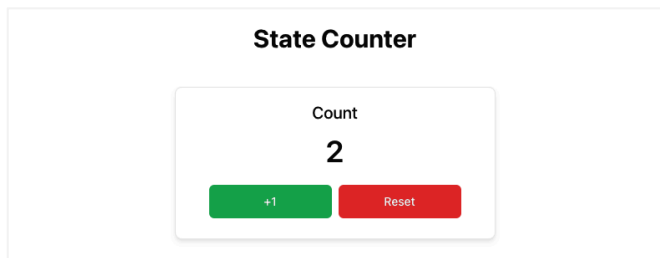


図 23: カウンター

「カウンターアプリですか。『+1』を押したら 1 が加算されて、『Reset』を押したら 0 に戻るわけですね」

「`create-vite` で生成されたプロジェクトにもついてる機能を、説明のために整理したものね。コンポーネントの `state` について理解するには、シンプルなカウンターがいちばんだと思う。最初に見るべきは `App.tsx` だけど、これは `Counter` コンポーネントをマウントしてるだけなので、その実体である `Counter.tsx` を開いてみよう」

リスト 37: `src/components/Counter.tsx`

```
import { useState } from 'react';
import { Button } from '@components/ui/button';
import { Card, CardContent, CardHeader } from '@components/ui/card';

function Counter() {
```

¹⁵⁸ <https://github.com/klemiary/Riakuto-StartingReact-ja5.0/tree/main/08-basics/03-state>

第9章 リンターとフォーマッターでコード美人に

9-1. リンターでコードの書き方を矯正する

9-1-1. リンターの選定

「ここまでコーディングスタイルを特に気にすることなくコードを書いてきた。でも本格的なアプリケーションを作る前にリンターを導入しておきたい」

「リンターって何ですか？」

「リンター (Linter) というのは一般的に、コードをチェック・解析してバグを生みにくい適切なコードの書き方になっているかとか、任意のコーディングスタイルに準拠しているかなどを検証してくれるツールのこと。一般的には『静的コード解析ツール』とも呼ばれる。『静的』というのはプログラムを実際には実行することなくって意味ね」

「そういえば Hooks API の使い方を学んだとき、`npm lint` というのを実行しましたよね」

「そう。create-vite で作成したプロジェクトはコードが Hooks の記述ルールに即しているかチェックするための Linter 環境が最初から用意されてるの。実際に実行されているのは `package.json` を見るとわかるけど、`eslint` というパッケージコマンドだね」

「ちなみに `lint` や `linter` って英語でどういう意味なんですか？」

「一般名詞で `lint` とは糸くずとか綿埃^{わたぼこり}のことね。ツールとしての最初のリンターは C 言語のソースコードに対して、コンパイラよりも詳細かつ厳密なチェックを行うプログラムとして開発され、初期の UNIX に `lint` コマンドとして装備されてた。その機能が、放っておくと故障の原因となる糸くずを片っぱしから絡めとる乾燥機の『`lint trap` (糸くずフィルター)』に似ていることから最初は『`linttrap`』と名付けるつもりだったんだけど、当時の UNIX では 5 文字以上のコマンドが作れなかったので『`lint`』と命名したんだって」

「へえ——」

「そこから転じて、C に限らず各種プログラミング言語で書かれたコードを解析して構文チェックを行うことを『`lint`』と動詞化して表現されるようになり、さらにそれを行うプログラムが『`linter`』と呼ばれるようになった。Ruby にも `RuboCop`¹⁸⁵ というリンターがあるよね。JavaScript で現在、もっとも広く使われているリンターが `ESLint`¹⁸⁶ ということなの」

「JavaScript のリンターなのに、`JSLint` じゃなく `ESLint` なんですね」

¹⁸⁵ <https://rubocop.org/>

¹⁸⁶ <https://eslint.org/>

「JSLint は別にあるよ。いや『あった』というべきかな。他のツールの例にもれず、JavaScript の世界ではリンターも流行り廃りや合従連衡が激しい。それらをざっと並べると次のようになる」

- **JSLint** (2002 年公開。最盛期は 2009 年～2012 年ごろ)
- **JSHint** (2011 年公開。JSLint からフォーク。最盛期は 2012 年～2015 年ごろ)
- **JSCS**¹⁸⁷ (2009 年公開。コーディングスタイルのチェックに特化したツール。2016 年に ESLint に吸収される)
- **ESLint**¹⁸⁸ (2013 年公開。2015 年ごろから急速にユーザー数を増やす)

「また TypeScript 専用の **TSLint**¹⁸⁹ というリンターがあったんだけど、色々あった末に 2019 年には typescript-eslint¹⁹⁰ という ESLint の TypeScript 対応プロジェクトに吸収された」

「つまり現在は JavaScript / TypeScript まとめて ESLint がデファクトスタンダードになってるわけですね」

「そう。ただここにも Rust 化の波が押し寄せてて、まだシェアは ESLint に及ぶべくもないんだけど着実に存在感を増してきてる。そのひとつが **Biome**¹⁹¹。もともと JavaScript のツールチェーンをそれひとつですべてカバーしようという Rome というプロジェクトがあったんだけど、道半ばで開発元の企業が破綻したために主要メンバーがフォークして立ち上げたという経緯がある¹⁹²」

「あー。終了した商用プロダクトをオープンソースコミュニティが引き継いで開発を続けるケースはたまに聞きますね」

「この Biome が 2024 年に入ってから急速にユーザー数を伸ばしつつある。Rome 同様、将来的には大統一ツールチェーンを目指しているものの、2025 年 4 月現在で実用化されている機能はリンターと、この後に説明するコードフォーマッターの 2 つ。

リンター機能における Biome の ESLint に対するアドバンテージは、まず Rust 製であることからくるパフォーマンスのよさ。自前のベンチマークでは ESLint と比べてマルチスレッドで 15 倍、シング

¹⁸⁷ <https://jscs-dev.github.io/>

¹⁸⁸ <https://eslint.org/>

¹⁸⁹ <https://palantir.github.io/tslint/>

¹⁹⁰ <https://typescript-eslint.io/>

¹⁹¹ <https://biomejs.dev/ja/>

¹⁹² 「JavaScript ツールチェーンの統一を目指した『Rome』のコアチームによる公式フォーク『Biome』が開始 - Publickey」
<https://www.publickey1.jp/blog/23/javascriptromebiome.html>

ルスレッドで4倍速いという結果になるらしい¹⁹³。プロジェクトの規模が大きくなってくると ESLint の実行にけっこう待たされるようになるから、この速さは魅力だね」

「ふむふむ」

「もうひとつは設定が超簡単なこと。後でやるけど ESLint は必要なプラグインをひとつひとつ組み込み、TypeScript に対応させるためにパーサーを導入して設定をして、また競合するルールを無効化する必要があったりと、ちゃんと動かすために必要な設定がものすごく複雑なの。いっぽう Biome はその必要がほとんどない。ESLint のような細かいカスタマイズはできないけど、ツールが統合されていてデフォルト設定が充実しているため、インストールしただけで85点の環境ですぐ動作する。これが ESLint の設定に疲れた人たちを惹きつけてるの」

「よさそう！ でも ESLint とどれくらい互換性があるんですか？」

「設定記法はもちろん、同じルールでも名前が ESLint と Biome ではちがうので純粋な意味での互換性はない。ただ機能としては ESLint の推奨ルールの8割ほどが Biome でも実装されてる。Biome の公式サイトに ESLint との対応表があるので参照してみても¹⁹⁴。React に関するルールは用意されてるけど、残念ながら TypeScript の型情報を必要とするルールがサポートされてない」

「そうなんですね」

「Rust 製のリンターでもうひとつ、**oxlint**¹⁹⁵を紹介しておこう。Vite の開発元である VoidZero による oxc プロジェクトで開発されているツールコレクションのひとつで、こちらは Biome のように ESLint と競合するのではなくその相互運用を見据えた設計になってる。ESLint の無効化コメント¹⁹⁶がそのまま使えたり、ESLint で実装されているものと同じ500以上のルールを同じ名前でサポートしてるの」

「へーえ。相互運用というのは具体的にはどういうことですか？」

「oxlint がサポートしているルールは oxlint に実行させ、それ以外を ESLint に実行させるといったことができる。そのために oxlint がサポートしているルールを ESLint で無効化するためのプラグインも提供してる¹⁹⁷。oxlint は自前のベンチマークで ESLint と比べて約50倍、Biome と比べても2倍以上は速いらしいので¹⁹⁸、大規模プロジェクトなら導入する価値はあるだろうね」

「50倍！ 爆速ですねえ」

¹⁹³ <https://github.com/biomejs/biome/tree/main/benchmark>

¹⁹⁴ 「Rules sources | Biome」
<https://biomejs.dev/linter/rules-sources/>

¹⁹⁵ 「Linter | The JavaScript Oxidation Compiler」 <https://oxc.rs/docs/guide/usage/linter.html>

¹⁹⁶ 「Configure Rules - ESLint - Pluggable JavaScript Linter」 > 「Disabling Rules」
<https://eslint.org/docs/latest/use/configure/rules#disabling-rules>

¹⁹⁷ <https://github.com/oxc-project/eslint-plugin-oxlint>

¹⁹⁸ <https://github.com/oxc-project/bench-javascript-linter>

「oxc プロジェクトでは TypeScript パーサーも開発されていて¹⁹⁹、oxlint で型情報を必要とするルールのサポートも開発中らしい。またロードマップには既存の ESLint プラグインのサポートも含まれてる²⁰⁰。将来的には ESLint をそのまま置き換えることができるようになるかもね」

「なるほど。それでウチのチームではどれを使うんですか？」

「うーん。小規模なプロジェクトなら Biome を実験的に導入してもいいかもしれないけど、通常の業務では当面はデファクトスタンダードである ESLint を使うことになるかな。構文チェックに時間がかかりすぎてメンバーがストレスを感じるような事態になれば、oxlint との相互運用を検討する感じ」

「ふむふむ」

「JavaScript はブラウザで稼働する言語として開発されたため後方互換性が捨てられず、理不尽な仕様がいつまでも残り続けてしまってる。古典的名著『JavaScript: The Good Parts』²⁰¹にあるような『悪いパーツ』を使わず『よいパーツ』によるベストプラクティスを強制するリンターの存在があって初めて、JavaScript はモダンな言語として完成される。ツールの栄枯盛衰はあれども、モダン開発における JavaScript にとってリンターは切り離せない言語の一部になっているの。この先は必ずプロジェクトに ESLint を導入していくからそのつもりでね」

「わかりました！」

9-1-2. ESLint の設定記法

「ESLint の環境構築においてありがちなのは、チームの誰かが作ってくれた設定に継ぎ足しを重ねて秘伝のタレみたいになってしまって、腐臭を放ちつつ肥大化していくパターン。各メンバーが内容を理解しないまま、動いてるからまあいいやとその場その場で設定を追加していくと、ついには誰にもメンテ不能になってしまう」

「腐臭を放つ秘伝のタレ……。リンターに限らず、チーム開発してるとよくある話ですよ」

「そうならないためには、メンバーひとりひとりが既存の内容を理解していて、随時メンテナンスを行っていく必要がある。だから秋谷さんにも一から ESLint の環境を構築し、編集できるようになってもらいたいの」

「はい、がんばります！」

¹⁹⁹ <https://oxc.rs/docs/guide/usage/parser.html>

²⁰⁰ 「oxlint Beta | The JavaScript Oxidation Compiler」
<https://oxc.rs/blog/2025-03-15-oxlint-beta.html>

²⁰¹ Douglas Crockford／水野貴明（訳）（2008）『JavaScript: The Good Parts —— 「よいパーツ」によるベストプラクティス』オライリー・ジャパン

「そのためにこれから ESLint の設定方法を学んでいくんだけど、その前にそれにまつわる大事な話をしておこう。ESLint は今、設定記法が大変革の時期にある。2024 年 4 月リリースの 9.0 において、初期から続いていた `eslintrc` 形式²⁰²が非推奨になり、新しく **Flat Config** という形式がデフォルトとなった。そして次のメジャーバージョンの 10.0 で `eslintrc` 形式は完全に削除される。

`create-vite` が生成するプロジェクトでの ESLint 環境も 2024 年 8 月リリースの 5.5 以降、`create-next-app` も 2024 年 12 月リリースの 15.1 以降は従来の `eslintrc` 形式から Flat Config に変更されてる。今後色々なプロジェクトが Flat Config になっていくはず」

「ええー、今はそんなめんどくさいタイミングなんですね……」

「ESLint の設定は 10 年以上ずっと `eslintrc` 形式だったから、膨大な数のプロジェクトがそれによって記述されてる。でも将来的に消えてしまう古い記法も同時におぼえろとはさすがに言えないので、今回は Flat Config 形式だけを教えるね」

「ありがとうございます。でもなぜ ESLint は設定記法をリニューアルなんてしたんですか？」

「ESLint は 10 年以上の歴史の中で、どんどん設定が複雑化し独自の構文を多く抱えることになってしまった。だから一度リセットして、JavaScript がわかる人なら誰でも読めるシンプルな構文にしたかったんだろうね」

「ふーむ。それで名前の Flat Config って、何がフラットなんですか？」

「`eslintrc` 形式ではプロジェクトのサブディレクトリにも設定ファイルを置くことができ、親ディレクトリから階層的に設定が適用されるようになってたの。Flat Config ではそれが廃止され、設定はプロジェクト全体で一元管理されるようになった。また外部の設定を継承するのではなく、JavaScript のオブジェクトとして読み込んでフラットに展開できるようになったことも由来してる」

「うーむ、わかったようなわからないような……」

「じゃ、実際に `create-vite` で作られた `eslint.config.js` ファイルを見ながら説明しよう」

リスト 42: `create-vite` が生成した `eslint.config.js`

```
import js from '@eslint/js'
import globals from 'globals'
import reactHooks from 'eslint-plugin-react-hooks'
import reactRefresh from 'eslint-plugin-react-refresh'
import tseslint from 'typescript-eslint'

export default tseslint.config(
  { ignores: ['dist'] },
  {
    extends: [js.configs.recommended, ...tseslint.configs.recommended],
    files: ['**/*.ts', '**/*.tsx'],
```

²⁰² 「Configuration Files (Deprecated) - ESLint - Pluggable JavaScript Linter」
<https://eslint.org/docs/latest/use/configure/configuration-files-deprecated>

```

languageOptions: {
  ecmaVersion: 2020,
  globals: globals.browser,
},
plugins: {
  'react-hooks': reactHooks,
  'react-refresh': reactRefresh,
},
rules: {
  ...reactHooks.configs.recommended.rules,
  'react-refresh/only-export-components': [
    'warn',
    { allowConstantExport: true },
  ],
},
},
)

```

「eslintrc 形式では CommonJS による JavaScript を始め JSON や YAML フォーマットでも記述できたんだけど、Flat Config では ESM のみとなっている。でも通常の JavaScript モジュールだから、JavaScript の構文で自由に変数を加工したりダンプしたりできる。たとえば

`reactHooks.configs.recommended.rules` の中身が知りたければ `console.dir()` で表示してやればいい。オブジェクトがネストしていて隠れる場合は第2引数に `{ depth: N }` オプションを指定すれば N 階層までが展開される」

「へー、そんなこともできるんですね」

「冒頭でいくつかライブラリがインポートされているよね。それらを順に説明すると次のようになる」

- `@eslint/js`²⁰³ Flat Config 用に ESLint 本体から切り離されたプラグイン
- `globals` ブラウザや Node.js などさまざまな環境でどのグローバル関数ができるか／使えないかが記述されている JSON ファイル
- `eslint-plugin-react-hooks`²⁰⁴ React で Hooks を使う際のルールを強制するプラグイン

²⁰³ <https://github.com/eslint/eslint/tree/main/packages/js>

²⁰⁴ <https://github.com/facebook/react/tree/main/packages/eslint-plugin-react-hooks>

- `eslint-plugin-react-refresh`²⁰⁵ React で開発サーバ起動中に Fast Refresh（コンポーネントの状態を失うことなく編集したコードの内容が反映される機能）が阻害されない書き方を強制するプラグイン
- `typescript-eslint`²⁰⁶ ESLint で TypeScript を扱えるようにする統合パッケージ。共有設定、パーサー、プラグインなどを含む

「ちょっと待ってください。プラグインとか共有設定とか、用語がよくわかりません」

「そうだね、ESLint における各種概念を整理しておこう」

- **プラグイン (Plugin)**

..... ESLint の組み込みルール以外に独自のルールを追加するもの。それらを適用した推奨の共有設定とパッケージングして提供されることが多い

- **パーサー (Parser)**

..... ソースコードを特定の言語仕様に沿って解析するためのライブラリ。ESLint には標準で JavaScript のパーサーが組み込まれているが、TypeScript には対応していないのでそのパーサーを導入する必要がある

- **共有設定 (Shareable Config)**

..... 複数ルールの適用をまとめて設定するもの。ESLint パッケージに同梱される推奨設定の `eslint:recommended` や `eslint-config-standard` などが有名

「プラグインってつまりは拡張ルールのことなんですね」

「正確には ESLint ではすべてのルールがプラグインとして設計されていて、コアルールも拡張ルールも等しくプラグインなんだけどね。ESLint の設定とは主に、どのプラグインを読み込んでどのルールをどのように適用するかを記述したものなの。ただそれらをひとつずつ記述するのは大変なので、通常はパッケージが提供してくれる推奨設定を適用したうえで部分的にカスタマイズする。このファイルでは `js.configs.recommended` と `tseslint.configs.recommended` の 2 つの共有設定を適用したうえで、React 用にカスタマイズを加えてるというわけ」

「なるほど」

「デフォルトエクスポートしているオブジェクトをいったん変数に格納してダンプすると、どんな条件でどのルールが適用されてるかが格納されたオブジェクトの配列を見ることができる。オブジェクトの間で設定が重複すれば、後ろのオブジェクトの設定値によって上書きされる。設定が複雑な依存

²⁰⁵ <https://github.com/ArnaudBarre/eslint-plugin-react-refresh>

²⁰⁶ <https://typescript-eslint.io/packages/typescript-eslint/>

関係を持たずフラットなので拡張性が高くメンテナンスしやすい。Flat Config という名前が納得できたんじゃないかな」

9-1-3. Flat Config で ESLint の設定を書く

「Flat Config の書き方を学ぶために、create-vite が作ってくれた `eslint.config.js` を拡張しやすい形に整理してみよう。わかりやすくするために、いったん React 用の設定は抜いておくよ。そのうえで次のように書き換える」

リスト43: 書き直した `eslint.config.js`

```
import globals from 'globals';
import pluginJs from '@eslint/js';
import tsEslint from 'typescript-eslint';
import { defineConfig } from 'eslint/config';

export default defineConfig([
  { files: ['**/*.js,mjs,cjs,ts,mts,cts,jsx,tsx'] },
  { ignores: ['{dist,build,public,node_modules}/**', '**/*.config.*'] },
  {
    languageOptions: {
      globals: {
        ...globals.browser,
        ...globals.es2025,
      },
    },
  },
  pluginJs.configs.recommended,
  tsEslint.configs.recommended,
]);
```

「まず設定のためのオブジェクト配列を生成するのに `typescript-eslint` パッケージが提供する `config()` 関数²⁰⁷を使うのをやめた。このヘルパー関数は `eslintrc` 形式にあったような `extends` プロパティを使うようにするものだけど、フラットで明示的な設定が損なわれてしまうので不要だと思う」

「代わりに `defineConfig()` という関数が使われてるようですけど？」

「代わりに入れたわけじゃないよ。これは型チェックヘルパーで、プロパティ名と値の組み合わせが適切ではないとコマンド実行時にエラーになる。残念ながらファイルが TypeScript じゃないので、VS

²⁰⁷ <https://typescript-eslint.io/packages/typescript-eslint#config>

Code の画面上では構文エラーを指摘してくれない」

「新しい `eslint.config.js` では `extends` プロパティがなくなってますけど、共有設定はどうやって読み込むんですか？」

「エクスポートする配列の中にただ置くだけでいい。さっきも言ったけど同じ設定項目がある場合は後ろのもので上書きされるので、順番には気をつけてね。なお配列の場合は自動的に展開されるので、`tsEslint.configs.recommended` は配列形式で提供されてるけど頭に `...` はつけてもつけなくても OK」

「だからこの記述ではついてないんですね」

「設定オブジェクトの中身を頭から見ていこう。`files` プロパティはそのまんま対象となるファイルを `glob` パターンで指定するもの。パターンは配列によって複数指定できる。`ignores` は逆にチェックの対象から除外するためのファイルの `glob` パターンを設定するものね」

「ところでなぜこれらはそれぞれ独立したオブジェクトとして配列に格納されてるんですか？ ひとつのオブジェクトの中のプロパティとして記述するほうが自然だと思うんですけど」

「Flat Config では `files` および `ignores` といった対象ファイルを指定するプロパティが含まれないオブジェクトの設定は、グローバルに有効になるの。逆に `files` か `ignores` のみを含むオブジェクトを配列に置くと、グローバルな設定がそのパターンで指定されたファイルに限定されるようになる。まずはグローバルな設定を先に作っておきたいのでこうしてるわけ」

「ふーむ、なるほど」

「`languageOptions` はその環境の JavaScript がサポートしている構文やグローバル変数を設定するプロパティ。元の `ecmaVersion` はなぜか 2020 だったけど、デフォルト値が `"latest"` なので削除した。グローバル変数の指定はブラウザのみだったので ES2025 を加えておいた。そして最後にインポートした 2 つの共有設定を置いている。`typescript-eslint` の共有設定には ESLint の共有設定からルールを上書きしている箇所があるので、必ず後ろに置くこと」

「これらの共有設定の内容はどうしたらわかるんでしょうか？」

「適用ルールについてはそれぞれ公式サイトにルール一覧表があって、Recommended なルールに ☒ マークがついてるのでそれでわかるはず²⁰⁸。設定全体を知りたい場合はコンソールにダンプすることだね。ちなみに `typescript-eslint` の推奨設定には以下のように TypeScript のパーサー設定が含まれてるために、自分で設定しなくても TypeScript が認識されるというのはおぼえておいてね」

```
{
  name: 'typescript-eslint/base',
  languageOptions: {
    parser: {
      meta: { name: 'typescript-eslint/parser', version: '8.30.1' },
```

²⁰⁸ <https://eslint.org/docs/latest/rules/> <https://typescript-eslint.io/rules/>

第 10 章 React をめぐるモダン Web 開発の歴史（後編）

10-1. 第 3 世代：SPA モデル黄金時代

10-1-1. React 登場以後の世界

「2024 年 12 月に安定版がリリースされた React 19 の新機能には、フレームワークと連携してサーバで運用することを前提としたものが少なくない。また以前に触れたように、現在の公式ドキュメントでは React の環境構築にはまず Next.js のようなフレームワークを使うことが推奨されている。React は SPA を構築するための UI ライブラリとして誕生したはずなのに、どういう経緯を経てこのような状況になったのかを説明しておこう。React 19 の新機能を使うためにフレームワークを導入する前に。中断してたモダン Web 技術の歴史の話はおぼえてる？」

「たしか jQuery に代表される第 1 世代、AngularJS に代表される第 2 世代があって、React の登場によって第 3 世代が始まったというところで終わってましたね」

「そう。React 公開以降に登場したフレームワークや UI ライブラリはその影響を受け、コンポーネントベースのアーキテクチャと単方向データフローの機構を備えるようになった。既存のフレームワークも多くがバージョンアップでその機能を取り入れた。これらをモダン Web 第 3 世代に分類できる」

「第 3 世代のフレームワークにはどんなものがあるんですか」

「数え上げればきりが無いんだけど、今でもそれなりのシェアを確保しているものに限定して挙げると Vue.js、Angular、Svelte、Preact²⁵⁶、Lit²⁵⁷、SolidJS²⁵⁸ あたりになるかな。何度も取り上げてる State of JavaScript の 2024 年の調査では、フロントエンドフレームワークの使用率は次のようになっている²⁵⁹」

²⁵⁶ <https://preactjs.com/>

²⁵⁷ <https://lit.dev/>

²⁵⁸ <https://www.solidjs.com/>

²⁵⁹ <https://2024.stateofjs.com/ja-JP/libraries/front-end-frameworks/>

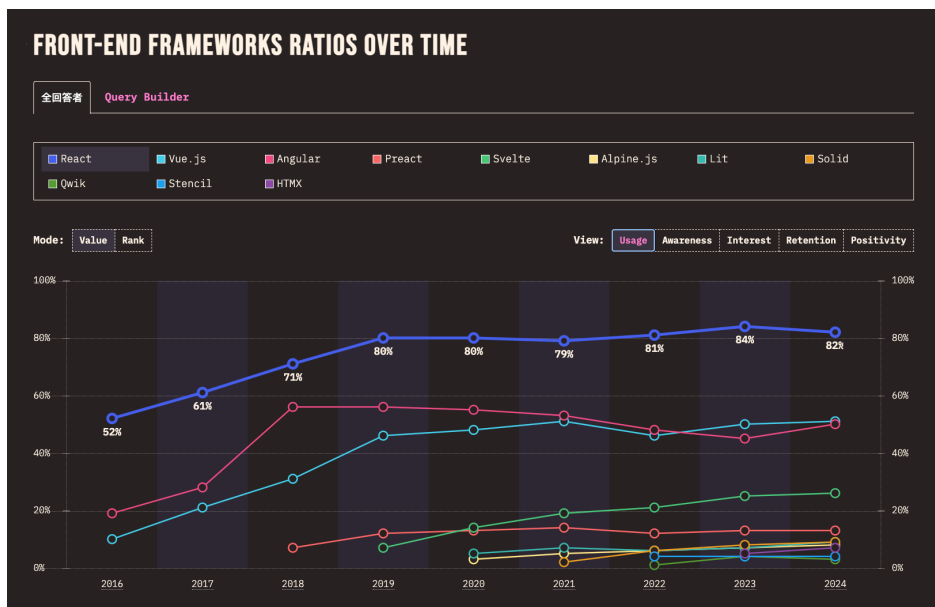


図 32: フロントエンドフレームワーク使用率 (State of JavaScript 2024 より)

「React が8割を超えていて圧倒的で、次に40～50%台の Vue.js と Angular、そしてその他大勢って感じですね」

「React、Angular、Vue.js は『三大フレームワーク』と称されることが多いけど、英語圏でも同様に『Big Three』と言われてる。ただ現状でも React と他2つの間には圧倒的な差がついてるうえ、Angular や Vue.js から React に乗り換える人は多くてもその逆はほとんどいないので、差が開く一方だけどね」

「やっぱりほとんど React 一強状態なんですね……」

「なぜそうなったかという考察も含めて、React 以外の第3世代の主なフロントエンドフレームワークをひとつずつ見ておこう」

10-1-2. Angular

「AngularJS と Angular を混同してる人が多いんだけど、実際は互換性のない別のフレームワークなのは知ってる？」

「えっ、知りませんでした。Angular はただ AngularJS を略して言うだけかと」

「Angular っていうのは AngularJS のチームがブランドとコンセプトを引き継いで一から新しく作り直したフレームワークで、当初は『Angular 2』を名乗ってたけど、バージョン3以降はただ Angular というようになった」

「ええー、そうだったんですか。なんてまぎらわしい……。けど、なぜ AngularJS はそんな互換性のないアップグレードをしたんですか？」

「AngularJS が双方向データバインディングを実現するために Dirty-Checking というしくみを利用して、そのせいでパフォーマンスに問題を抱えてた話はしたよね。その後に仮想 DOM と単方向データフローによってパフォーマンスの問題を解決した React が登場して、本格的な SPA を開発したい人たちが AngularJS から乗り換える動きが目立つようになった。だから AngularJS の開発チームは、古臭くなってしまったアーキテクチャを改善するより新しいパラダイムで作直したほうがいいと考えたんだろうね」

「ふむ。それまで一番人気の座にあったのに、乗り換える人が続出したら焦っちゃう気持ちはわかります」

「しかしこれはかなり大がかりなプロジェクトだった。2014 年の 10 月にその計画を発表して、2.0 の安定版が出たのは 2016 年 9 月。作業に丸 2 年もかかったことになる」

「動きの早いフロントエンドの世界でそれは痛いですね。ほぼ作り直したったんでしょうけど、Angular って AngularJS とそんなに別物なんですか？」

「踏襲してるのは HTML テンプレートと DI（依存性注入）のしくみを持つことくらいで、あとは共通点を見つけるのが難しいほどだね。\$scope や Controller の概念がなくなり、コンポーネントベースのアーキテクチャになった。もっとも思い切った変更は TypeScript を全面採用するようになったこと。JavaScript でも開発できないけども、ドキュメントも不足してるし推奨されてない。これは Angular 2 に移行した動機である大規模開発に対応させるため。なお当初は AtScript という JavaScript に静的型付け機能を加えた言語を自前で開発していたところを、Microsoft の TypeScript チームからの働きかけで TypeScript を採用することにしたという逸話がある。TypeScript がかなり早い時期からデコレータ²⁶⁰を備えていたのは、この移行時の Angular チームからの強い要望によるものらしいね」

「へーえ。それで刷新された Angular は、既存のユーザーから受け入れられたんですか？」

「成功したとはいいい難いね。まず TypeScript の採用は長期的にはまちがってはいなかったけど、当時としては急進的すぎた。AngularJS はそのとっつきやすさでライトユーザーに人気だったのに、JavaScript を厳格にした別の言語を学ばないとバージョンアップできないというのは、ライトユーザーが離反する要因になった。また大規模開発への対応のため、オブジェクト指向を本格的に導入したこともハードルを上げることになった」

「Angular、本格的に触ったことはないですけど、なんか Java っぽいというかエンタープライズ臭が強いなというイメージはあります」

「また Angular は至れり尽くせりのフルスタックフレームワークだったことが、変化の早いフロント

²⁶⁰ クラスおよびそのメソッドやプロパティにメタデータを追加する方法を提供する構文。Angular ではクラスをコンポーネントやモジュールなどに定義するために常用される。
<https://www.typescriptlang.org/docs/handbook/decorators.html>

エンドの世界では災いした。フロントエンドの世界では日々新しいニーズが生まれる中で、それに対応してさまざまなライブラリやツールが作られ、それらが激しい競争の中で淘汰・洗練されていく。シンプルな View ライブラリだった React はそこで勝ち残った技術を組み合わせることでトレンドに対応できたけど、すべてを公式で用意しようとする Angular はその波に取り残されてしまった」

「Rails ではフルスタックなことが強みだったのに、フロントエンドではそれが逆に悪いほうに働いてしまうとは……」

「残念なことに Angular の DX に対するユーザーの評価は高くない。State of JavaScript 2023 のフロントエンドフレームワークに対する『感情的に肯定的か否定的か』という調査では、React が肯定：否定が 7:3 なのに対して Angular ではそれがほぼ逆転してる。この数字は他のフレームワークと比べても圧倒的に悪い²⁶¹」

「このグラフを見ると、既存のユーザーの間ですら否定が上回ってるのは Angular だけですね……」

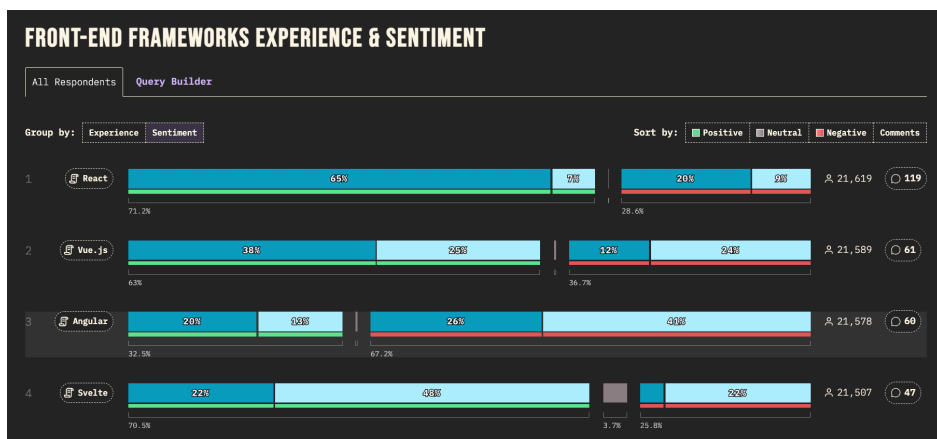


図 33: フレームワークに対して肯定的か否定的か (State of JavaScript 2023 より)

「そういう事情からか、npm パッケージのダウンロード数で 2022 年初旬に Vue.js に追い抜かれてしまった。状況を打開するために 2023 年 11 月のバージョン 17 で、ロゴの変更と公式サイトのリニューアルを伴う大改修を実施。新しいテンプレート記法を導入、サーバサイドレンダリングや静的サイト生成などの機能を盛り込んだ。ただこれも下降トレンドを打開するには至っておらず、2024 年には npm ダウンロード数が Preact にも抜かれてしまった」

「え、ビッグスリーの座から実質的に転落してるんですか。モダン Web の世界は栄枯盛衰がハンパないですね……」

²⁶¹ https://2023.stateofjs.com/ja-JP/libraries/front-end-frameworks/#front_end_frameworks_experience

10-1-3. Vue.js

「Google で AngularJS を使った開発に携わっていたひとりのエンジニア、Evan You が『AngularJS で冗長に感じられる部分を切り捨て必要な部分だけを抽出して、もっと軽いものを作ることができないだろうか』と考えた。そうして彼は AngularJS から HTML テンプレートを、KnockoutJS から MVVM パターンによるデータバインディングを、React からコンポーネントベースのアーキテクチャを取り入れた軽量なフレームワークを開発した。それに view のフランス語に相当する vue という名前をつけて最初のバージョンである 0.8 を 2014 年 2 月に公開した」

「なんか悪く言えば節操がない、よく言えば既存の技術のいいところ取りなフレームワークだったんですね」

「そのあたりのバランス感覚というか、ポジション取りは絶妙だったと言えるだろうね。とっつきやすかった AngularJS から一気にハードルの上がった Angular 2 へのアップグレードについていけず、かといって React の関数型プログラミング的アプローチにも魅力を感じないライト層をうまく取り込んでいった。特に日本では、私が本格的に React に取り組み始めた 2016 年ごろから 3 年間くらいは、Vue.js の勢いが React を上回ってた」

「Rails エンジニアが多い日本は、テンプレート形式の Vue に惹かれる人が多かったんでしょうね」

「ちなみにビッグスリーで Angular が Google、React が Meta という大企業の後ろ盾がある中、Vue.js はずっとコミュニティベースのプロジェクトとして継続してるのは特筆すべきことだと思う。Evan You はツールチェーンの Vite というもう 1 本のホームランも打ってるので、ユーザーが望んでいることに対する嗅覚と、それをプロダクトに反映させるバランス感覚が優れた開発者なんだろうね。特にバージョン 1.0 で導入された、ひとつのファイルの中にロジックとテンプレート、スタイルをまとめて書ける SFC（Single-File Components）はコミュニティの枠を超えて絶大な支持を得て、他のフレームワークにも影響を与えた」

リスト 60: Vue2 での SFC のサンプルコード

```
<script>
module.exports = {
  data: () => ({
    greeting: 'Hello',
  }),
};
</script>

<template>
  <p class="greeting">{{ greeting }} World!</p>
</template>
```

```
<style scoped>
.greeting {
  color: red;
  font-weight: bold;
}
</style>
```

「ああ、これ Vue のチュートリアルで見ました。SFC っていうんですか。わかりやすくいいですね」

「ただそんな彼でも、長年プロジェクトを続けているうちに『一から作り直したい病』にかかってしまった」

「一から作り直したい病？ Angular チームがかかったみたいですか？」

「そう。Vue.js はバージョン 2 系からバージョン 3 系へのバージョンアップで互換性を捨てて一から作り直したの。プロジェクトが長期化すればどうしてもレガシーコードが蓄積されていくし、TypeScript へのサポートの構造的な限界や、React が宣言的 UI を完成の域に高めつつあることなどに焦りを感じたのかもしれない。そうして 2020 年 9 月にバージョン 3.0 の正式版がリリースされた」

「2 系と互換性がないって、何がそんなに変わったんでしょうか？」

「色々あるけど、一番の変更は Composition API の導入。それまでコンポーネントのロジックの記述には Options API というものが使われてた。初学者にはとっつきやすいものではあったものの、ロジックがコンポーネントに密結合しがちで、分離・再利用しづらいという短所があった。だからそれを完全に捨てて新しく Composition API を作ったわけ。そしてこの Composition API、React ユーザーが一見すればわかるけど、完全に React の Hooks API をアレンジして移植したものなのね」

「いいところ取りが持ち味の Vue だから、今回も React からそれを取り入れたんですね」

「しかしその経緯が全く異なる。React は 2015 年と比較的早くからコンポーネントを関数で定義できるようになっていたものの、関数コンポーネントに状態や副作用を追加する手段がなかったため、実質的にはほとんどのコンポーネントをクラスで定義するしかない状況が長く続いた。

そんな状況でコミュニティから発生したさまざまな手法が試行錯誤されていた中、公式から Hooks API という待望の最適解が提示されたため、開発者は喜んで移行したわけ」

「なるほど。Vue はそういう背景がなかったのに、作者の一存でそんなものを導入しちゃったわけですか」

「そう。関数型プログラミングの素養がなく、テンプレートで書きやすいからという理由で Vue.js を使っていたライト層は、どういう脈絡でそんな難易度の高いものが導入されたのか理解に苦しんだらうね。ところで Vue.js では同じ機能を持つ付属パッケージが 2 系では vue-template-compiler、3 系では @vue/compiler-sfc の名前で登録されているので、その 2 つのダウンロード数を比較することで移行状況が推測できる」

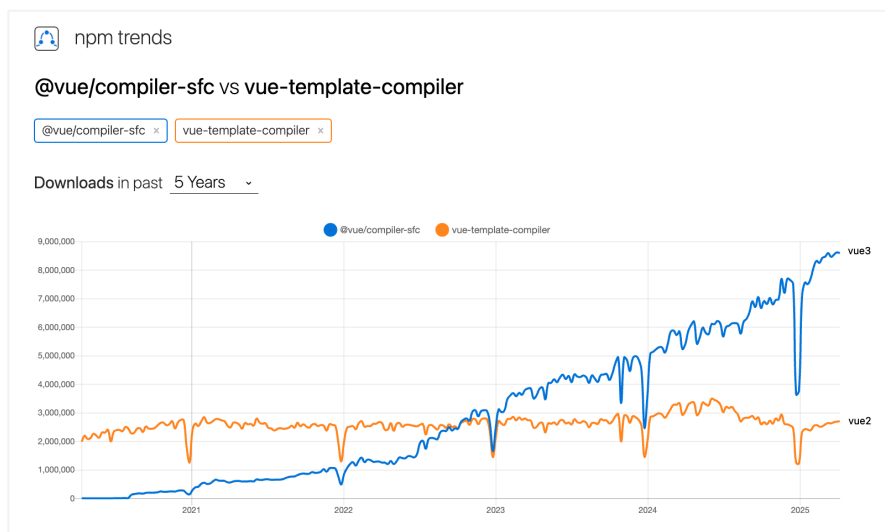


図 34: Vue2 と Vue3 のダウンロード数比較（npm trends, Apr. 2025）

「これを見ると、3 系が 2 系のそれに追いつくまでに丸 2 年もかかっている。そして 2 系の公式サポートは 2023 年いっぱいでは終了しているにもかかわらず、ダウンロード数は 2024 年半ばまで増え続けた。また移行コストが高いため、Composition API で書き直すくらいなら React に移行しようというケースが企業を中心に増えていった。日本でも Vue.js が React に逆転されたのは Vue 3 がリリースされた 2020 年から 2021 年にかけてじゃないかな」

「あらー、Angular と同じ轍を踏んじゃったわけですか……」

「Vue.js コアチームは『React でできることは Vue でもできるのに、なぜ Vue を試してみる人が少ないのだろう?』と疑問に感じてらしいけど、React のほうが圧倒的に普及してるのにわざわざ機能的に後追いの Vue を使う人はいないと思う。ライト層ならともかくそこそこ熟練したプログラマが純粋な JavaScript の関数の中で完結してしまう世界を経験すれば、テンプレート記法と関数型プログラミングが中途半端に組み合わせられたものに魅力は感じないよね」

「柴崎さんらしい意見ですね」

「ただ作者の Evan You が中国系で中国の開発者コミュニティと密接な関係にあることから、中国でのシェアは Vue.js が React を圧倒してる。Nuxt²⁶² という強力なタグパートナーの存在も手伝って、今後も React に次ぐ 2 位の座をキープしそうだね」

²⁶² <https://nuxt.com/>

10-1-4. Svelte

「秋谷さんは Vue.js の SFC を直感的でわかりやすいと言ってたよね。Svelte はそれをさらに推し進めたシンプルな構文で書けるフレームワーク。公式サイトにブラウザ上で Svelte を手軽に試せる REPL があるので²⁶³、簡単なコードを書いてみよう。ちなみにファイルの拡張子は `.svelte` という独自フォーマットになる」

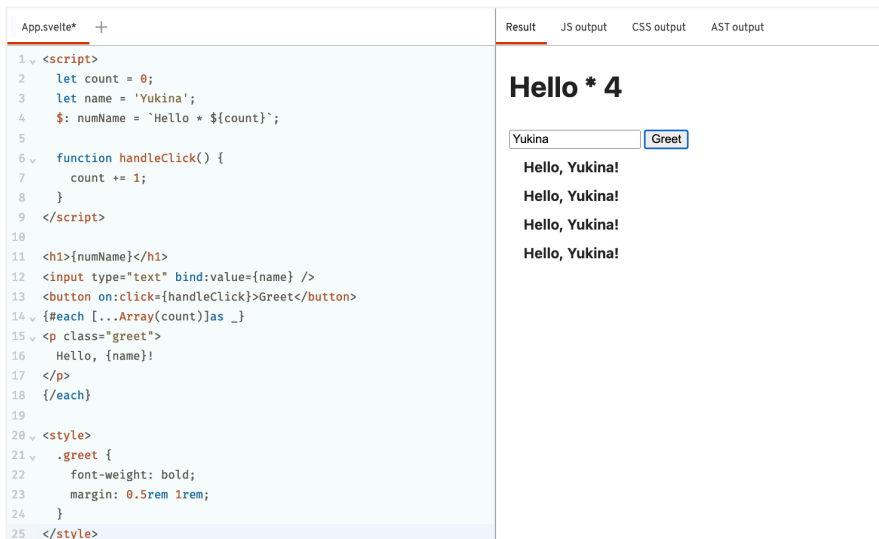


図 35: Svelte のコードサンプル

「`<script>`」の中にロジックを、`<style>`の中に CSS を書くのは SFC と共通してますけど、テンプレートは直書きなんですね。テンプレートの構文も Vue よりずっと素直でおぼえやすそう。あとちょっとびっくりしたのは、`count` はただ普通に `let` で宣言しただけの変数なのに、双方向バインディングできちゃうんですね」

「Svelte の公開当初の一番の売りが『Write less code』だったからね。おまじないめなものは徹底的に書かなくて済むようになってる。だからか State of JavaScript のフレームワークのリテンション率（継続使用率）ランキングでも登場時からずっと 1 位か 2 位という好順位につけてる²⁶⁴」

²⁶³ <https://svelte.dev/repl>

²⁶⁴ <https://2024.stateofjs.com/ja-JP/libraries/front-end-frameworks/>

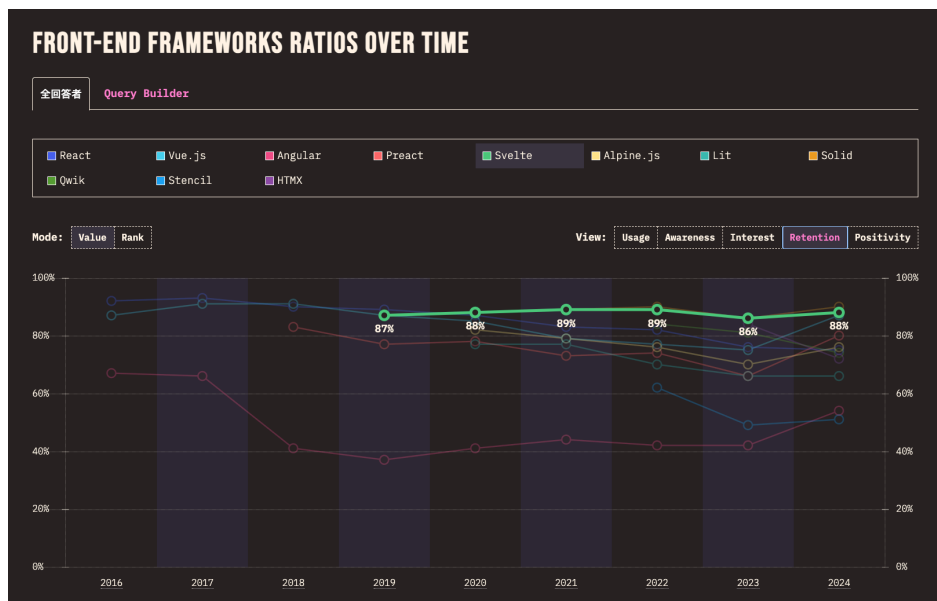


図 36: Svelte のリテンション率の推移（State of JavaScript 2024 より）

「なんか使ってみたくなくなってきました。ちなみに Svelte って『スヴェルテ』じゃなく『スヴェルト』って読むんですね。何語ですか？」

「もともとフランス語だけど、輸入されて英語にもなってる。すらりとした細さとか、洗練されたしなやかさを形容する言葉で、slender をより上品にしたようなニュアンスらしい。

Svelte の特徴はそれだけじゃなく、非常にパフォーマンスが高いフレームワークでもある。Svelte で開発したプロジェクトは、ビルドするとランタイムを含まない純粋な JavaScript ファイルが生成されるの。React だと、ビルド後のファイルには React と React DOM のパッケージが含まれていて、アプリケーションの機能はそれを介して実現されるわけだけど、Svelte にはそれが無い。だからバンドルサイズも小さく、パフォーマンスにも有利に働く」

「へ——」

「だから Svelte は当初、自らをフレームワークでも UI ライブラリでもなく『コンパイラ』と称してた。Svelte の作者 Rich Harris はモジュールバンドラの Rollup や DOM 操作ライブラリの Ractive.js²⁶⁵ のオリジナル作者でもある。そういう立場にいたことで、宣言型に定義されたコンポーネントをコン

²⁶⁵ <https://ractive.js.org/>

パイラが解釈し、最適化されたJavaScript コードに変換するという発想に至ったんだって²⁶⁶」

「ふむふむ。コードの記述量も少なくて済むし、パフォーマンスもすばらしい。でも知名度的にはそれほどでもないですね。私も知らなかったし。もっと使われててもよさそうなのに」

「当初はコンパイルを挟むために実行時のエラーがわかりにくい、後発のためエコシステムが小さく対応するライブラリが少ない、書き方に制約が少ないのでチーム開発でコード品質にばらつきが生じやすいといった不満を感じて離れていったユーザーもいたみたいね。でもバージョンアップを重ねるごとに、こういった点もかなり解消されてきてる。著名企業での採用例も作者の勤務先である New York Times を始め、Apple Music や IKEA など増えつつある²⁶⁷」

「けっこう実績があるんですね。それで柴崎さんの予想としては、この先 Svelte はもっと伸びそうですか？ Vue を脅かす存在になる？」

「うーん、どうなんだろうね。Svelte は 2024 年 10 月リリースの 5.0 から Rune という新しい構文が導入された²⁶⁸。コンポーネントの状態や副作用をもっと明示的に書けるようにして、これまで Svelte が苦手としていた大規模開発にも対応できるようにしたもので、いわば Svelte 版の Hooks API だね。私が見る限りシンプルにまとまって、グローバルな状態管理なんかはむしろこれまでよりシンプルに書けるようになってる」

「問題はライト層がどう思うかですよ。Angular 2 や Vue 3 のように離反を招きかねないのでは？」

「バージョンアップに関してこれまで Svelte はうまくやって、内部的にオーバーホールするようなときでも構文の下位互換性は維持してた。古くなってきた機能もいきなり捨てずにまず非推奨を勧告し、段階的に廃止するようにしてる。このへんは React の戦略を見習ってるのかもね。だから Rune が導入されても以前の書き方は残されるだろうし、それで既存のユーザーが離れていくことはないと思う。Svelte は学習コストの低さ、パフォーマンスの高さにおいて Svelte がフロントエンドフレームワークの中で群を抜いた存在なのはまちがいない。エコシステムが Svelte 5 に追いつけば、かなり有力な選択肢になってくると思う」

²⁶⁶ 「JavaScript 言語にリアクティビティを実現するフロントエンドフレームワーク Svelte 3 - 作者の Rich Harris 氏に聞く - InfoQ」
<https://www.infoq.com/jp/news/2019/07/svelte-3-interview-rich-harris/>

²⁶⁷ 「Top 10 Big Companies Using Svelte - Okupter」
<https://www.okupter.com/blog/companies-using-svelte>

²⁶⁸ 「Rune 導入 (Introducing runes)」<https://svelte.jp/blog/runes>

10-2. 第 4 世代：メタフレームワークの時代

10-2-1. バック・トゥ・ザ・サーバ

「Web アプリケーションの歴史は MPA に始まり、jQuery に代表されるモダン Web 第 1 世代によって動的かつインタラクティブな UX を導入した。しかしそれによって難易度が格段に上がってしまった開発をよりシンプルで簡単にするために第 2・第 3 世代の技術によって SPA へと移行してきたというのはこれまで説明したとおり。しかし冷静に振り返ってみると第 2 世代以降の進化はあくまで開発者の都合であり、アプリケーションのユーザーにとって直接メリットになることは何もなかった。DX が 1→2→3 世代で大幅に向上したおかげで Web 上にインタラクティブなアプリケーションの量は増えたかもしれないけど、アプリケーションのユーザーが享受できる質的なメリットについては 2005 年の Google マップから基本的に変わってない」

「.....言われてみればそうですね」

「むしろ悪化した面もある。Web アプリケーションが MPA から SPA になったことで動的かつインタラクティブな UX をスーパーエンジニアでなくても作れるようになったわけだけども、その代償として多くのデメリットを抱え込むことになった。考えつくものをざっと挙げてみると、まずアプリケーションを使うユーザーにとっては次のようなものがある」

- 初回表示までに時間がかかる

..... SPA は初回ロード時にアプリケーション全体の JavaScript コードをダウンロードして実行する必要があるため、最初にアクセスしてから画面が表示されるまでの時間が長い

- SEO に弱い

..... ページが静的な HTML ではなく JavaScript によって動的に DOM が生成される形式のため、検索エンジンのクローラが頻繁に訪問しなかったり、コンテンツの解析が適切になされなかったりして順位付けに不利になりがち

- クライアントの負荷が高い

..... レンダリングを始めとするほとんどのタスクがブラウザで実行されるため、クライアントのメモリや計算リソースを多く消費する。また、そもそも JavaScript が無効な環境では動作しない

「けっこうありますね。これだけ一斉を風靡したんだから SPA 化っていいことばかりのように思えてましたけど、全然そうじゃなかったんだ」

「でもこれらのデメリットは、SPA 黄金時代において DX 向上の大義名分のもとに軽視されてきた。ただ少数ながら警鐘を鳴らす人たちもいて、Chrome チームの初代テックリードで Web Components の提唱者である Alex Russel はその最先鋒だった。彼は『JavaScript は Web の CO₂ だ²⁶⁹』『ショッピ

²⁶⁹ 「The “Developer Experience” Bait-and-Switch - Infrequently Noted」
<https://infrequently.org/2018/09/the-developer-experience-bait-and-switch/>

第3部「React実践編」へつづく

著者紹介

大岡由佳（おおおか・ゆか）

国内の IT 企業数社での PHP や Ruby による開発およびプロダクトマネージャを経て、2016 年からフリーランスに転身。当時まだ日本ではマイナーな存在だった React に惚れ込み、React 専門のフリーランスエンジニアとして複数の現場を渡り歩く。その経験を元に執筆した「りあクト！」シリーズが評判を呼びヒット作となる。現在は事実上おそらく国内（世界でも？）に唯一生息する、専門の技術同人作家。

趣味は 3.5 倍速で聴くオーディオブックの読書と、ジム代わりに 7 年続いているバレエ。

X アカウントは @oukayuka。ブログは <https://klemiwary.com>（くるみ割り書房公式サイト）。

イラスト：黒木めぐみ（くろき・めぐみ）

漫画家、イラストレーター。

「りあクト！」シリーズの表紙イラストを一貫して担当。

りあクト！ TypeScript で始めるつらくない React 開発 第 5 版

【① 言語・環境編】



現場のエンジニアから多大な支持を受ける『りあクト！ TypeScript で始めるつらくない React 開発』の最新 5 版、全 3 部構成の第 1 部「言語・環境編」。

モダンな Web 開発において JavaScript および TypeScript のスキルは大事な基礎体力。その基礎を固めつつ、実際の React 開発において押さえておきたい仕様やさらに進んだ書き方までを学んでいきます。JS や TS が初めての方はもちろん、初～中級者の方にもオススメです。

BOOTH にて絶賛販売中！

(2025 年 5 月 30 日発行／B5 判・214p／電子版 ¥1,200／紙＋電子セット ¥1,550)

【目次】

- 第 1 章 こんにちは React
- 第 2 章 ライトでディープな JavaScript の世界
- 第 3 章 関数型プログラミングでいこう
- 第 4 章 TypeScript で型をご安全に

りあクト！ TypeScript で始めるつらくない React 開発 第 5 版

【③ React 実践編】



現場のエンジニアから多大な支持を受ける『りあクト！ TypeScript で始めるつらくない React 開発』の最新 5 版、全 3 部構成の第 3 部「React 実践編」。

19 ではサーバ稼働が前提の機能が追加され、もはや React はフレームワークで使われるのが当然の時代。そこで第 5 版では SPA に加えて Nex.js および React Router（旧 Remix）での各種機能の使い方を丁寧に紹介しています。

React 未経験者にはもちろん、19 の新機能やフレームワークについて学びたい初～中級者の方にもオススメです。BOOTH にて絶賛販売中！

（2025 年 5 月 30 日発行／B5 判・215p／電子版 ¥1,200／紙＋電子セット ¥1,550）

【目次】

- 第 11 章 React のページをルーティングする
- 第 12 章 外部リソースのデータを取得する
- 第 13 章 エラーをハンドリングする
- 第 14 章 メモ化でレンダリングを効率化する
- 第 15 章 フォームでデータを更新する

メールマガジン登録のご案内



くるみ割り書房 BOOTH ページ

「りあクト！」シリーズを刊行している「くるみ割り書房」では、新刊や紙の本の再版予定、その他読者の方へのお知らせなどをメールマガジン形式で不定期に配信しています。

購読をご希望の方は、「くるみ割り書房 BOOTH」で検索して最初に表示された上記画像のページにて、「フォロー」ボタンからサークルのフォロー登録をお願いします。BOOTH の一斉送信メッセージにてメルマガの内容をお届けします。

(※ BOOTH のアカウントが必要になります。BOOTH はピクシブ株式会社が運営するオンラインショップです)

りあクト！ TypeScript で始めるつらくない React 開発 第5版

【② React 基礎編】

2025 年 5 月 30 日	初版第 1 刷発行
2025 年 5 月 30 日	電子版バージョン 1.1.0

著	者	大岡 由佳	
発	行	元	くるみ割り書房
連	絡	先	oukayuka@gmail.com

本書を無断で複写・複製、転載、データ配信、オークションやフリマアプリに出品することを固く禁じます。

© 2025 Yuka O'oka / Klemiuary Books